

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC
KỸ THUẬT PHÁT TRIỂN ỨNG DỤNG WEB
ĐỀ TÀI: WEBSITE MẠNG XÃ HỘI TÍCH HỢP
BLOCKCHAIN

Môn học: IE213.Q21

Giảng viên hướng dẫn: ThS. Võ Tấn Khoa

Thực hiện bởi nhóm 12, bao gồm:

1. Nguyễn Trọng Nhân	22521004	Trưởng nhóm
----------------------	----------	-------------

Thời gian thực hiện: 06-02-2026 – 12-05-2026

MỤC LỤC

MỤC LỤC	2
DANH SÁCH HÌNH	6
DANH SÁCH BẢNG.....	7
TÓM TẮT.....	8
Chương 1 Tổng quan	11
1.1 Giới thiệu đề tài	11
1.1.1 Lý do chọn đề tài	11
1.1.2 Mục tiêu đề tài	12
1.1.3 Phạm vi đề tài	12
1.1.4 Đối tượng người dùng	13
1.2 Khảo sát hệ thống tương tự	14
1.3 Đóng góp của đề tài	15
1.4 Bố cục báo cáo.....	16
Chương 2 Cơ sở lý thuyết.....	18
2.1 MERN stack	18
2.2 Công nghệ blockchain	19
2.2.1 Khái niệm	19
2.2.2 Hàm băm Keccak-256	19
2.2.3 Smart Contract.....	19
2.3 Ethereum và Sepolia testnet	20
2.3.1 Máy ảo Ethereum (EVM).....	20
2.3.2 Vòng đời giao dịch	20
2.3.3 Gas – Cơ chế phí	21
2.3.4 Mạng thử nghiệm Sepolia	21
2.4 Solidity và OpenZeppelin.....	22
2.4.1 Ngôn ngữ Solidity	22
2.4.2 Thư viện OpenZeppelin.....	22
2.5 Thư viện Ethers.js	23
2.6 Bảo mật smart contract.....	24

2.6.1 Tần công Reentrancy	24
2.6.2 Mô hình CEI (Checks-Effects-Interactions).....	24
2.6.3 Pull payment so với Push payment.....	25
2.6.4 Một số rủi ro khác.....	25
2.7 Công nghệ phụ trợ	26
2.7.1 JSON Web Token (JWT)	26
2.7.2 Cloudinary	26
2.7.3 Socket.io	27
2.7.4 Firebase Authentication	27
2.7.5 Mongoose	27
2.7.6 AWS.....	27
Chương 3 Phân tích và thiết kế hệ thống.....	28
3.1 Phân tích yêu cầu	28
3.1.1 Yêu cầu chức năng.....	28
3.1.2 Yêu cầu phi chức năng	29
3.2 Kiến trúc tổng thể	30
3.3 Thiết kế backend.....	31
3.3.1 Kiến trúc phân tầng Controller – Service – DAO	31
3.3.2 Authentication và authorization.....	34
3.3.3 Realtime với socket.io	35
3.4 Thiết kế frontend	36
3.5 Thiết kế cơ sở dữ liệu	37
3.5.1 Các thực thể chính	38
3.5.2 Các thực thể phụ	39
3.6 Thiết kế smart contract	40
3.6.1 ContentRegistry — Đóng dấu Bài đăng.....	40
3.6.2 Charity — Quyên góp theo Cột mốc.....	41
3.6.3 So sánh BE-relay và FE-signed pattern.....	44
3.7 Bảo mật smart contract.....	45
Chương 4 Cài đặt và triển khai.....	47
4.1 Môi trường phát triển.....	47

4.2 Cài đặt backend	49
4.2.1 Cấu trúc thư mục và tổ chức module.....	49
4.2.2 Một module tiêu biểu — Charity.....	50
4.2.3 Quy ước throw lỗi — <code>AppError</code> và <code>errorHandler</code>	52
4.2.4 Cron job và xử lý thời gian.....	53
4.3 Cài đặt frontend	55
4.3.1 Cấu trúc thư mục và lazy routing	55
4.3.2 <code>Axios</code> instance và xử lý 401.....	56
4.3.3 Context Web3 — Quản lý ví và chuyển mạng	56
4.4 Cài đặt Smart Contract	57
4.4.1 <code>ContentRegistry.sol</code>	57
4.4.2 <code>Charity.sol</code>	59
4.4.3 Triển khai lên Sepolia	60
4.5 Tích hợp blockchain	61
4.5.1 Flow đăng bài và đóng dấu (<code>ContentRegistry</code>).....	61
4.5.2 Flow donate và record (<code>Charity</code>).....	63
4.5.3 Off-chain cache pattern	65
Chương 5 Kiểm thử và đánh giá.....	69
5.1 Kiểm thử Smart Contract.....	69
5.1.1 Phương pháp và công cụ	69
5.1.2 Phân nhóm các test case	69
5.1.3 Test reentrancy – điểm trọng tâm	70
5.1.4 Báo cáo gas	71
5.1.5 Đánh giá độ bao phủ kiểm thử	72
5.2 Tối ưu hiệu năng frontend	72
5.2.1 Phương pháp đo	72
5.2.2 Bottleneck phát hiện ở baseline.....	73
5.2.3 Các kỹ thuật tối ưu áp dụng.....	74
5.2.4 Kết quả sau tối ưu.....	75
5.3 Phản hồi từ Workshop review.....	76
5.3.1 Bối cảnh workshop	76

5.3.2 Tổng hợp phản hồi.....	76
5.3.3 Các cải tiến đã thực hiện sau workshop	78
5.3.4 Tổng kết phản hồi workshop	79
5.4 Đánh giá blockchain	79
5.4.1 ContentRegistry – Đóng dấu bài đăng	80
5.4.2 Charity – Quyên góp theo cột mốc.....	81
5.4.3 So sánh tổng quát với Web2	82
Chương 6 Kết luận và Hướng phát triển	83
6.1 Kết quả đạt được.....	83
6.2 Hạn chế	83
6.3 Hướng phát triển.....	84
NGUỒN THAM KHẢO	85

DANH SÁCH HÌNH

Hình 1 Mã nguồn minh họa AccessControl và ReentrancyGuard trong Charity.sol ...	23
Hình 2 Minh họa áp dụng mô hình CEI (Checks-Effects-Interactions).....	25
Hình 3 Sơ đồ kiến trúc tổng thể của SocialApp	30
Hình 4 Kiến trúc phân tầng Controller – Service – DAO của backend	32
Hình 5 Cấu trúc thư mục mã nguồn frontend.....	36
Hình 6 Sơ đồ lồng nhau của các React Context provider.....	37
Hình 7 Sơ đồ ERD các collection chính của cơ sở dữ liệu	38
Hình 8 Interface và lưu trữ của smart contract ContentRegistry.....	40
Hình 9 Sáu trạng thái của Charity Campaign.....	41
Hình 10 Cấu trúc thư mục backend	49
Hình 11 Cấu trúc file router charity.route.js	51
Hình 12 Controller mở rộng đóng vai trò router HTTP.....	51
Hình 13 Validation chống lừa đảo trong charityService.js.....	52
Hình 14 Class AppError thay cho Error chuẩn.....	52
Hình 15 Middleware errorHandler phân nhánh theo cờ isOperational	53
Hình 16 Cron job charityExpiryCron.js	53
Hình 17 Đoạn mã chính của cron job đánh dấu chiến dịch quá hạn	54
Hình 18 Cấu trúc thư mục mã nguồn frontend.....	55
Hình 19 Khai báo lazy import cho các page trong App.jsx.....	55
Hình 20 Hai interceptor của Axios instance	56
Hình 21 Xử lý các trạng thái của ví Web3 trong Web3Context	57
Hình 22 Mã nguồn smart contract ContentRegistry.sol	58
Hình 23 Khai báo kế thừa của contract Charity.sol.....	59
Hình 24 Năm event phục vụ off-chain index và verify	59
Hình 25 Hàm claimRefund áp dụng pattern Checks-Effects-Interactions	60
Hình 26 Script triển khai deploy-charity.js.....	60
Hình 27 Sơ đồ luồng đăng bài và đóng dấu qua ContentRegistry	61
Hình 28 Trích đoạn fire-and-forget trong postService.js.....	63
Hình 29 Sơ đồ luồng quyên góp và ghi nhận qua Charity	63
Hình 30 Đoạn mã parse event để xác thực giao dịch on-chain	65
Hình 31 Trích đoạn hàm syncChainCache đồng bộ cache off-chain	66
Hình 32 Endpoint POST /api/charity/campaigns/:id/sync	67
Hình 33 Cấu trúc thư mục kiểm thử	69
Hình 34 Câu lệnh chạy test.....	69
Hình 35 Mã nguồn ReentrancyAttacker.sol	71
Hình 36 Test Reentrancy	71
Hình 37 Áp dụng lazy cho import DonateModel	75
Hình 38 Network hint preconnect trong public/index.html	75

DANH SÁCH BẢNG

Bảng 1 Danh sách từ viết tắt.....	9
Bảng 2 Các nhóm người dùng và quyền hạn.....	13
Bảng 3 So sánh các hệ thống tương tự với SocialApp	14
Bảng 4 Các nhóm chức năng và use case chính	28
Bảng 5 Yêu cầu phi chức năng	29
Bảng 6 Trách nhiệm các tầng kiến trúc Backend	33
Bảng 7 Sáu trạng thái của Charity Campaign	41
Bảng 8 Bốn vai trò của AccessControl.....	42
Bảng 9 Quy tắc bảo vệ donor (chống lừa đảo).....	43
Bảng 10 So sánh pattern BE-relay và FE-signed	44
Bảng 11 Defense in depth cho các quy tắc kinh doanh quan trọng.....	45
Bảng 12 Các công cụ và phiên bản sử dụng.....	47
Bảng 13 Danh sách các api backend	49
Bảng 14 Mapping state on-chain và cache off-chain cho Charity.....	66
Bảng 15 Cấu hình triển khai production.....	67
Bảng 16 Phân nhóm và liệt kê test case của Charity.sol	69
Bảng 17 Gas cost trung bình các function Charity.sol	71
Bảng 18 Số liệu Lighthouse mobile (baseline — trước tối ưu).....	73
Bảng 19 Áp dụng Cloudinary URL transformation tại các vị trí render ảnh	74
Bảng 20 Số liệu Lighthouse mobile (sau tối ưu).....	75
Bảng 21 Phản hồi tích cực từ các nhóm review	76
Bảng 22 Bốn điểm trừ chung được nhắc đến	77
Bảng 23 Bug nghiêm trọng được phát hiện trong workshop	78
Bảng 24 So sánh giá trị cốt lõi giữa Web2 và blockchain trong đề tài.....	82

TÓM TẮT

Đề tài SocialApp xây dựng một mạng xã hội đầy đủ tính năng theo kiến trúc MERN (MongoDB, Express, React, Node.js) và tích hợp công nghệ blockchain Ethereum để giải quyết hai vấn đề thực tế của mạng xã hội Web2 truyền thống: chống chỉnh sửa nội dung lên thông qua việc đóng dấu băm bài đăng lên blockchain, và tăng tính minh bạch cho hoạt động quyên góp từ thiện bằng smart contract giải ngân theo cột mốc.

Hệ thống gồm ba thành phần chính: backend Node.js / Express tổ chức theo kiến trúc phân tầng Controller - Service - DAO với 15 module nghiệp vụ; frontend React 19 sử dụng Context API cho quản lý trạng thái và lazy route cho code-splitting; và hai smart contract Solidity được triển khai trên mạng thử nghiệm Sepolia

— ContentRegistry lưu hash bài đăng theo pattern BE-relay (backend trả gas, người dùng không cần ETH) và Charity cho phép quyên góp theo pattern FE-signed (donor tự ký giao dịch để bảo toàn msg.sender). Việc phân biệt rõ hai pattern tích hợp blockchain khác nhau cho hai use case khác nhau là một đóng góp chuyên môn của đề tài.

Để đảm bảo bảo mật và tính tin cậy, contract Charity áp dụng các thư viện bảo mật của OpenZeppelin (AccessControl, ReentrancyGuard) cùng pattern Checks-Effects-Interactions và pull-payment. Bộ kiểm thử gồm trên 40 test case bao quát các luồng hoạt động bình thường, biên, sai phép, và đặc biệt là tấn công reentrancy với một mock contract chuyên dụng. Quy tắc chống lừa đảo (tối thiểu 2 cột mốc, mỗi cột mốc tối đa 50% mục tiêu, tổng đúng bằng mục tiêu) được enforce ở ba lớp (Solidity, backend service, frontend form) theo nguyên tắc defense in depth.

Hệ thống được triển khai thực tế tại tên miền medicine.id.vn cho phép demo end-to-end. Sau khi nhận phản hồi từ buổi workshop nội bộ với 7 nhóm review, đề tài đã sửa 8 bug nghiêm trọng, bổ sung 4 file README cho từng phần, vẽ 6 sơ đồ kiến trúc và sequence cho báo cáo, đồng thời tối ưu hiệu năng frontend bằng kỹ thuật transformation Cloudinary URL, lazy import các modal Web3 và preconnect CDN. Kết quả Lighthouse trên thiết bị di động tăng từ 70 lên 79 điểm cho trang /charity và LCP giảm 37%, từ 6977 ms xuống 4362 ms.

Từ khóa: Mạng xã hội, Blockchain, Smart Contract, MERN, Solidity, Ethereum, Sepolia, Quyên góp minh bạch, Đóng dấu nội dung, Reentrancy, React, Lighthouse.

Bảng 1 Danh sách từ viết tắt

Viết tắt	Cụm đầy đủ	Giải thích ngắn
ABI	Application Binary Interface	Đặc tả định dạng giao tiếp với smart contract
API	Application Programming Interface	Giao diện lập trình ứng dụng
AVIF	AV1 Image File Format	Định dạng ảnh thế hệ mới, nén tốt hơn WebP
BE	Backend	Phần máy chủ của ứng dụng
CDN	Content Delivery Network	Mạng phân phối nội dung tĩnh
CEI	Checks-Effects-Interactions	Pattern chống reentrancy trong smart contract
CLS	Cumulative Layout Shift	Chỉ số đo độ ổn định layout, thuộc Web Vitals
CRA	Create React App	Toolchain mặc định của React
CRUD	Create-Read-Update-Delete	Bốn thao tác cơ bản trên dữ liệu
CSDL	Cơ sở dữ liệu	–
DAO	Data Access Object	Lớp truy cập dữ liệu trong kiến trúc phân tầng
DOM	Document Object Model	Cây cấu trúc của tài liệu HTML
EOA	Externally Owned Account	Tài khoản Ethereum có khóa riêng tư
ERC	Ethereum Request for Comments	Tiêu chuẩn token Ethereum (ví dụ ERC-20, ERC-721)
EVM	Ethereum Virtual Machine	Máy ảo thực thi bytecode Ethereum
FCP	First Contentful Paint	Thời điểm vẽ nội dung đầu tiên, thuộc Web Vitals
FE	Frontend	Phần giao diện ứng dụng
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
HTTPS	HTTP Secure	HTTP có mã hóa TLS
JWT	JSON Web Token	Định dạng token xác thực không trạng thái
LCP	Largest Contentful Paint	Thời điểm vẽ phần nội dung lớn nhất, thuộc Web Vitals

Viết tắt	Cụm đầy đủ	Giải thích ngắn
MERN	MongoDB, Express, React, Node.js	Stack công nghệ JavaScript đầy đủ
MXH	Mạng xã hội	–
NFT	Non-Fungible Token	Token độc nhất, thường dùng cho tài sản số
OZ	OpenZeppelin	Thư viện smart contract chuẩn công nghiệp
RBAC	Role-Based Access Control	Mô hình phân quyền dựa trên vai trò
REST	Representational State Transfer	Phong cách kiến trúc API
RPC	Remote Procedure Call	Lời gọi hàm từ xa, dùng để giao tiếp với node Ethereum
SDK	Software Development Kit	Bộ công cụ phát triển
SPA	Single-Page Application	Ứng dụng một trang, render bằng JavaScript
SSR	Server-Side Rendering	Render HTML phía máy chủ
TBT	Total Blocking Time	Tổng thời gian thread chính bị chặn, thuộc Web Vitals
TLS	Transport Layer Security	Giao thức bảo mật tầng vận chuyển
TTL	Time To Live	Thời gian sống của bản ghi
URL	Uniform Resource Locator	Địa chỉ tài nguyên trên web
VPS	Virtual Private Server	Máy chủ ảo riêng
WebP	Web Picture format	Định dạng ảnh nén tốt do Google phát triển

Chương 1 Tổng quan

1.1 Giới thiệu đề tài

1.1.1 Lý do chọn đề tài

Mạng xã hội (MXH) đã trở thành hạ tầng giao tiếp quan trọng của xã hội số hiện đại. Theo thống kê của We Are Social, đến đầu năm 2025, trên thế giới có hơn 5,2 tỷ tài khoản mạng xã hội đang hoạt động — tương đương khoảng 63% dân số toàn cầu [1]. Tại Việt Nam, các nền tảng như Facebook, Instagram, TikTok hay Zalo gắn liền với hoạt động hằng ngày của hầu hết người dùng Internet ở mọi độ tuổi.

Tuy nhiên, các mạng xã hội Web2 truyền thống đang bộc lộ hai vấn đề ngày càng được người dùng và giới nghiên cứu quan tâm:

Thứ nhất, vấn đề toàn vẹn nội dung. Khi mọi bài đăng, bình luận và dấu thời gian đều được lưu trong cơ sở dữ liệu trung tâm của nhà cung cấp dịch vụ, người dùng không có cách nào tự kiểm chứng rằng nội dung họ đã đăng chưa bị chỉnh sửa âm thầm bởi đội ngũ vận hành hoặc bị tấn công xâm nhập sửa đổi. Nhiều trường hợp tranh chấp pháp lý về quyền tác giả hoặc tuyên bố trên mạng xã hội đã vướng phải khó khăn trong việc xác minh thời điểm gốc của nội dung.

Thứ hai, vấn đề tính minh bạch của các hoạt động quyên góp trực tuyến. Các nền tảng gây quỹ truyền thống (như GoFundMe ở thị trường quốc tế) hoặc các trang quyên góp do tổ chức tự dựng đều yêu cầu nhà tài trợ đặt niềm tin hoàn toàn vào ban tổ chức rằng tiền sẽ được sử dụng đúng mục đích. Thực tế đã có nhiều trường hợp gian lận được báo chí đưa tin, trong đó người tổ chức thu tiền rồi không thực hiện cam kết, gây mất niềm tin của cộng đồng vào hoạt động từ thiện trực tuyến.

Công nghệ blockchain, với các đặc tính bất biến và minh bạch, là một lời giải tự nhiên cho cả hai vấn đề trên. Một bài đăng có thể được "đóng dấu" bằng hash nội dung lên blockchain — bất kỳ ai sau này cũng có thể tự kiểm chứng nội dung và thời điểm mà không cần tin máy chủ. Một chiến dịch quyên góp có thể vận hành bằng smart contract khóa tiền và chỉ giải ngân theo các cột mốc đã được công khai từ đầu — nhà tài trợ không cần tin tổ chức "không cầm tiền chạy" mà có thể tự kiểm tra trên Etherscan.

Đề tài SocialApp được chọn với hai mục đích đan xen. Về mặt học thuật, đề tài là cơ hội để vận dụng đầy đủ kiến thức MERN Stack — kết hợp với công nghệ blockchain Ethereum vào một sản phẩm có quy mô tương đối và có tính ứng dụng. Về mặt thực tiễn, đề tài chứng minh rằng việc tích hợp blockchain vào ứng dụng mạng xã hội không nhất thiết phải làm phức tạp toàn bộ trải nghiệm: blockchain chỉ được áp dụng đúng chỗ — hai use case yêu cầu tính phi tập trung (không đặt niềm tin tuyệt đối vào

nhà cung cấp dịch vụ) — còn các tính năng còn lại (đăng bài, bình luận, kết bạn, chat) vẫn vận hành theo mô hình MERN truyền thống nhanh và rẻ.

1.1.2 Mục tiêu đề tài

Đề tài hướng đến ba mục tiêu cụ thể:

Mục tiêu 1 — Xây dựng một mạng xã hội có quy mô tính năng tương đương một sản phẩm Web2 thực tế. Hệ thống cần triển khai đầy đủ các nhóm chức năng cốt lõi mà người dùng kỳ vọng ở một MXH hiện đại: đăng ký / đăng nhập (bao gồm cả Google OAuth và đăng nhập bằng ví Web3), đăng bài có ảnh / video, bình luận lồng nhau, like, kết bạn, theo dõi, story 24 giờ, chat 1-1 và chat nhóm theo thời gian thực, gọi thoại, thông báo realtime, nhóm cộng đồng, lưu bài (bookmark) và tìm kiếm.

Mục tiêu 2 — Tích hợp hai smart contract Solidity giải quyết hai use case:

- ContentRegistry — đóng dấu băm nội dung mọi bài đăng lên Sepolia testnet. Người dùng và bên thứ ba có thể verify nội dung gốc và thời điểm đăng mà không cần tin máy chủ SocialApp.
- Charity — quản lý các chiến dịch quyên góp theo cột mốc. Tiền ETH bị khóa trong contract, chỉ giải ngân khi quản trị viên duyệt báo cáo của tổ chức ngoài chuỗi. Nếu chiến dịch hết hạn không đủ mục tiêu hoặc bị tranh chấp, nhà tài trợ tự gọi claimRefund để nhận lại tiền.

Mục tiêu 3 — Triển khai sản phẩm thực tế trên môi trường production để có thể demo end-to-end cho hội đồng đánh giá. Hệ thống phải chạy ổn định trên một tên miền thật với chứng chỉ TLS, kết nối đến Sepolia testnet, và xử lý được các luồng nghiệp vụ phức tạp như đăng bài có đóng dấu on-chain, quyên góp ETH, hoàn tiền khi thất bại, và quản trị các chiến dịch.

1.1.3 Phạm vi đề tài

Trong phạm vi:

- Hai smart contract đã liệt kê ở mục 1.1.2, được kiểm thử kỹ và triển khai trên mạng thử nghiệm Sepolia.
- Toàn bộ tính năng mạng xã hội cốt lõi đã liệt kê.
- Triển khai production trên AWS EC2 (backend) và Vercel (frontend) với tên miền medicine.id.vn.
- Bộ kiểm thử tự động cho smart contract bao gồm cả tấn công reentrancy.
- Tài liệu kỹ thuật đầy đủ: README cho từng phần, sáu sơ đồ kiến trúc, báo cáo cuối kỳ.

Ngoài phạm vi:

Nhằm bảo đảm chất lượng các tính năng đã có thay vì dàn trải, một số tính năng blockchain phức tạp đã được chủ động loại khỏi phạm vi triển khai, bao gồm: token ERC-20 nội bộ (SocialToken) và cơ chế tipping bằng token cho người sáng tạo nội dung, NFT thành viên (MembershipPass), NFT bài đăng (PostNFT) và chợ trao đổi vật phẩm trong nhóm cộng đồng (GroupMarketplace). Các tính năng trên không đáp ứng thời gian và cũng không thực sự cần thiết để chứng minh năng lực tích hợp blockchain vì hai smart contract ContentRegistry và Charity đã đủ thể hiện hai pattern thiết kế khác biệt; riêng GroupMarketplace đòi hỏi logic escrow phức tạp vượt quá quy mô phù hợp của một đồ án môn học.

- *Vòng cắt thứ nhất (ngày 18/03/2026):* loại bỏ bốn tính năng — token ERC-20 nội bộ (SocialToken), tipping bằng token cho người sáng tạo nội dung, NFT thành viên (MembershipPass), và NFT bài đăng (PostNFT). Bốn tính năng này tuy thú vị nhưng không cần thiết để chứng minh năng lực tích hợp blockchain — hai contract ContentRegistry và Charity đã đủ thể hiện hai pattern khác biệt.
- *Vòng cắt thứ hai (ngày 27/03/2026):* loại bỏ tính năng GroupMarketplace — chợ trao đổi vật phẩm trong nhóm cộng đồng. Tính năng này tuy đã có thiết kế nhưng đòi hỏi thêm một smart contract nữa và logic escrow phức tạp, không khả thi trong thời gian còn lại của học kỳ.

Đề tài cũng không sử dụng mainnet Ethereum vì chi phí gas thực và không phù hợp với mục đích học tập. Việc chuyển sang mainnet hoặc Layer 2 (Arbitrum, Base), nhóm em sẽ nghiên cứu và phát triển thêm trong thời gian tới.

1.1.4 Đối tượng người dùng

Hệ thống phục vụ bốn nhóm tác nhân với các quyền hạn khác nhau:

Bảng 2 Các nhóm người dùng và quyền hạn

Tác nhân	Quyền chính
Người dùng không có metamask	Đăng / xem bài, bình luận, like, kết bạn, theo dõi, chat, đăng story, lưu bài, tham gia nhóm.
Người dùng có metamask	Giống với người dùng phổ thông, nhưng có thể liên kết ví MetaMask để quyên góp ETH
Chủ tổ chức	Toàn bộ quyền của User, cộng thêm: nộp đơn xin xác minh tổ chức, tạo và quản lý chiến dịch quyên góp sau khi được admin

Tác nhân	Quyền chính
	duyet. Một tài khoản chỉ có thể sở hữu tối đa một tổ chức đang hoạt động.
Quản trị viên (Admin)	Toàn bộ quyền của User. Có thể duyệt đơn xin xác minh tổ chức (kèm theo tác whitelist ví on-chain), duyệt báo cáo cột mốc của chiến dịch để giải ngân, ép trạng thái thất bại cho chiến dịch khi có tranh chấp, và quản lý các báo cáo vi phạm cộng đồng.

1.2 Khảo sát hệ thống tương tự

Để định vị đóng góp của đề tài, mục này so sánh SocialApp với bốn hệ thống đại diện trên thị trường, được phân thành hai trục: trục công nghệ (Web2 thuần vs có tích hợp blockchain) và trục lĩnh vực (mạng xã hội vs quyên góp).

Bảng 3 So sánh các hệ thống tương tự với SocialApp

Hệ thống	Lĩnh vực	Công nghệ	Điểm mạnh	Điểm yếu mà SocialApp giải quyết
Facebook, Instagram	MXH	Web2	UX hoàn thiện; quy mô tỷ người dùng; nhiều tính năng	Admin có thể sửa nội dung và timestamp âm thầm; người dùng không tự kiểm chứng được
GoFundMe	Quyên góp	Web2	UX đơn giản, dễ tiếp cận với người không hiểu công nghệ	Người tổ chức cầm tiền hoàn toàn; nhiều trường hợp lừa đảo đã được báo chí ghi nhận
Gitcoin Grants	Quyên góp	Web3 (Ethereum mainnet)	Minh bạch on-chain; có cơ chế Quadratic Funding	Bắt buộc người dùng có ví và ETH; phí gas mainnet đắt; UX phức tạp với người mới

Hệ thống	Lĩnh vực	Công nghệ	Điểm mạnh	Điểm yếu mà SocialApp giải quyết
Mirror.xyz	Xuất bản nội dung	Web3 (Optimism / Arbitrum)	Đóng dấu bài viết on-chain; có hỗ trợ NFT cho bài viết	Không có tính năng tương tác xã hội (kết bạn, chat, nhóm)

Phân tích bảng trên cho thấy, không có nền tảng nào đồng thời cung cấp đầy đủ tính năng mạng xã hội Web2 hiện đại, tích hợp blockchain đúng chỗ cho hai use case yêu cầu tính phi tập trung, và giữ được trải nghiệm thân thiện với cả người dùng chưa có ví Web3. Các nền tảng Web2 thiếu tính minh bạch ở những chỗ cần; các nền tảng Web3 hiện có lại quá chuyên biệt và không phục vụ tốt nhu cầu mạng xã hội nói chung.

Đề tài SocialApp định vị ở khoảng giữa: kế thừa UX và phạm vi tính năng của Web2, áp dụng blockchain có chọn lọc cho hai vấn đề nội dung và quyền góp, với hai pattern tích hợp khác nhau (BE-relay và FE-signed) tương ứng với đặc thù của từng use case. Đây là một hướng tiếp cận Blockchain 2.0 có tính thực dụng cao và hoàn toàn khả thi để triển khai trong khuôn khổ một đề án môn học.

1.3 Đóng góp của đề tài

Đề tài có bốn đóng góp chính, được sắp theo thứ tự từ thực tiễn đến chuyên môn:

Đóng góp 1 — Một sản phẩm hoàn chỉnh có thể demo end-to-end. SocialApp được triển khai thực tế tại medicine.id.vn với khoảng 25 tính năng nghiệp vụ thuộc 9 nhóm (Auth, Social, Realtime, Group, Story, Save, Organization, Charity, Admin). Hai smart contract đã được verify trên Etherscan, có thể được kiểm chứng độc lập bởi bất kỳ ai có địa chỉ contract.

Đóng góp 2 — Phân biệt rõ hai pattern tích hợp blockchain. Đề tài trình bày và minh họa bằng hai contract thực tế hai pattern khác biệt:

- ContentRegistry áp dụng pattern BE-relay — backend ký giao dịch bằng ví của hệ thống và trả gas thay người dùng. Phù hợp cho các thao tác miễn phí mà không cần ràng buộc msg.sender.
- Charity áp dụng pattern FE-signed — người dùng tự ký giao dịch từ ví của họ. Bắt buộc khi contract enforce định danh msg.sender, ví dụ để ghi nhận đúng địa chỉ donor cho việc hoàn tiền sau này.

Việc lựa chọn đúng pattern là quyết định kiến trúc quan trọng nhưng ít được thảo luận rõ ràng trong các tài liệu nhập môn về tích hợp blockchain.

Đóng góp 3 — Bộ kiểm thử smart contract bao gồm tấn công reentrancy. Đề tài cung cấp trên 40 test case Hardhat / Chai cho contract Charity, trong đó có hai test case sử dụng một mock contract chuyên dụng (ReentrancyAttacker.sol) tái hiện cơ chế tấn công của The DAO 2016. Bộ test này có thể dùng làm tham khảo cho các đề tài blockchain khác.

Đóng góp 4 — Defense in depth ba lớp cho bảo vệ donor. Các quy tắc chống lừa đảo (tối thiểu 2 cột mốc, mỗi mốc tối đa 50% mục tiêu, tổng đúng bằng mục tiêu) được enforce ở ba tầng độc lập: Solidity (require, không bypass được), backend service (validate trước khi gọi contract), frontend form (UX cảnh báo ngay khi nhập). Cách tiếp cận này thể hiện nguyên tắc không đặt niềm tin tuyệt đối vào bất kỳ tầng nào của bảo mật ứng dụng — kể cả khi backend bị bypass, contract vẫn từ chối các campaign sai cấu trúc.

1.4 Bố cục báo cáo

Báo cáo gồm sáu chương cùng phần đầu (bìa, lời cảm ơn, lời cam đoan, tóm tắt, danh sách hình bảng), tài liệu tham khảo và phụ lục. Tóm tắt nội dung từng chương như sau:

- Chương 1 — Tổng quan. Giới thiệu đề tài, mục tiêu, phạm vi, đối tượng người dùng; khảo sát các hệ thống tương tự; tóm tắt đóng góp của đề tài và bố cục báo cáo.
- Chương 2 — Cơ sở lý thuyết. Trình bày các kiến thức nền tảng cần thiết để hiểu các chương sau, bao gồm: kiến trúc MERN, công nghệ blockchain, mạng Ethereum và Sepolia, ngôn ngữ Solidity, thư viện OpenZeppelin, Ethers.js, các pattern bảo mật smart contract (CEI, pull-payment, reentrancy), và một số công nghệ phụ trợ.
- Chương 3 — Phân tích và Thiết kế hệ thống. Phân tích yêu cầu chức năng và phi chức năng, đặc tả use case. Trình bày kiến trúc tổng thể của hệ thống, thiết kế chi tiết phần backend (kiến trúc phân tầng), frontend (component, state, routing), cơ sở dữ liệu (sơ đồ ERD), và đặc biệt là thiết kế của hai smart contract cùng so sánh hai pattern tích hợp.
- Chương 4 — Cài đặt và Triển khai. Mô tả môi trường phát triển, cấu trúc thư mục từng phần, các đoạn mã đại diện, luồng tích hợp blockchain end-to-end (đăng bài + đóng dấu, quyên góp + ghi nhận), pattern off-chain cache, và cấu hình triển khai production trên AWS/Vercel .

- Chương 5 — Kiểm thử và Đánh giá. Trình bày bộ kiểm thử smart contract với hơn 40 test case bao gồm tấn công reentrancy; quy trình đo và tối ưu hiệu năng frontend bằng Lighthouse với số liệu trước / sau cụ thể; tổng hợp phản hồi từ buổi workshop review nội bộ và các cải tiến đã thực hiện; đánh giá định tính giá trị của blockchain trên hai tiêu chí "minh bạch" và "hữu ích" cho từng contract.
- Chương 6 — Kết luận và Hướng phát triển. Tổng kết các kết quả đã đạt được, phân tích các hạn chế còn tồn tại, đề xuất các hướng mở rộng trong tương lai (di chuyển sang Next.js để có SSR, multi-sig admin, DAO voting, Layer 2), và rút ra các bài học cá nhân trong quá trình thực hiện đề tài.

Chương 2 Cơ sở lý thuyết

2.1 MERN stack

MERN là tên viết tắt của một bộ công nghệ phổ biến cho phát triển ứng dụng web full-stack, gồm bốn thành phần đều dựa trên ngôn ngữ JavaScript: MongoDB, Express, React và Node.js [2]. Lựa chọn MERN cho đề tài đáp ứng đúng yêu cầu của môn học, đồng thời tận dụng được hai ưu điểm thực tế của bộ công nghệ này: ngôn ngữ đồng nhất xuyên suốt máy chủ và máy khách, và hệ sinh thái thư viện npm phong phú nhất trong các ngôn ngữ hiện nay.

MongoDB là cơ sở dữ liệu phi quan hệ thuộc nhóm document store, lưu dữ liệu ở định dạng BSON (Binary JSON). Khác với cơ sở dữ liệu quan hệ truyền thống yêu cầu schema cố định trước, MongoDB cho phép schema mềm dẻo — phù hợp với các thực thể có cấu trúc phát triển nhanh như bài đăng trên mạng xã hội. Các thao tác chính là find, insert, update, delete trên các collection. MongoDB hỗ trợ các tính năng quan trọng cho ứng dụng quy mô vừa: index B-tree và TTL, replica set cho khả dụng cao, aggregate pipeline cho truy vấn phức tạp, change stream cho realtime, và driver chính thức cho hầu hết các ngôn ngữ.

Express là framework HTTP tối giản chạy trên Node.js, triết lý thiết kế là "không gò bó". Express xoay quanh khái niệm middleware — một hàm có chữ ký (req, res, next) được mount theo thứ tự, mỗi middleware xử lý một phần nhỏ của một request. Toàn bộ tính năng từ parse JSON, xác thực người dùng, kiểm soát tốc độ truy cập, đến xử lý lỗi đều có thể được gói thành middleware. Express phiên bản 5 (phát hành chính thức cuối năm 2024) là phiên bản được sử dụng trong đề tài, có nhiều cải tiến về xử lý async error so với phiên bản 4.

React là thư viện front-end để xây dựng giao diện người dùng dựa trên component. Mỗi component là một hàm JavaScript trả về cây JSX (cú pháp giống XML mở rộng cho JavaScript) mô tả phần giao diện cần render. React duy trì một bản sao của cây DOM trong bộ nhớ (Virtual DOM) và chỉ cập nhật những phần thay đổi thực sự lên DOM thật, giúp hiệu năng render tốt hơn so với thao tác DOM thủ công. Phiên bản 19 được sử dụng trong đề tài giới thiệu khái niệm Server Component và một số cải tiến về Suspense cho lazy loading.

Node.js là môi trường runtime JavaScript phía máy chủ, được Ryan Dahl giới thiệu năm 2009. Node.js dùng máy ảo V8 của Google (cùng máy ảo của Chrome) để thực thi JavaScript ngoài trình duyệt. Đặc điểm kiến trúc nổi bật của Node.js là event loop đơn luồng kết hợp với non-blocking I/O dựa trên thư viện libuv: thay vì tạo một thread mỗi khi có request như mô hình truyền thống, Node.js xử lý mọi I/O ở chế độ bất đồng bộ và thông báo lại qua callback, hợp với các ứng dụng có nhiều thao tác I/O

nhưng ít tính toán nặng — đúng với đặc thù của một mạng xã hội. Để tận dụng tối đa CPU, đề tài sử dụng PM2 ở chế độ cluster cho deploy production

2.2 Công nghệ blockchain

2.2.1 Khái niệm

Blockchain là một cấu trúc dữ liệu dạng danh sách liên kết, trong đó mỗi phần tử (gọi là block) chứa một hoặc nhiều giao dịch, một dấu thời gian, và một con trỏ tới block trước được biểu diễn bằng giá trị băm (hash) của block đó. Việc thay đổi nội dung của một block bất kỳ trong chuỗi sẽ làm hash của block đó thay đổi, kéo theo phải tính lại hash của toàn bộ các block phía sau — chi phí tính toán này, kết hợp với cơ chế đồng thuận phân tán giữa nhiều node độc lập, khiến việc gian lận trở nên không khả thi trong thực tế. Hai thuộc tính quan trọng của blockchain là bất biến (immutability) và minh bạch (transparency): bất kỳ ai có quyền truy cập một node đầy đủ đều có thể đọc và xác minh toàn bộ lịch sử giao dịch.

2.2.2 Hàm băm Keccak-256

Trên Ethereum, hàm băm được sử dụng phổ biến nhất là Keccak-256 [3]. Đây là một biến thể được Ethereum áp dụng trước khi NIST chuẩn hóa thành SHA-3 vào năm 2015 — hai hàm tuy có cùng cấu trúc tổng quát nhưng khác nhau về padding, do đó kết quả băm không giống nhau. Đặc điểm của hàm băm mật mã được dùng cho blockchain bao gồm:

- Tính tất định — cùng một đầu vào luôn tạo ra cùng một đầu ra, bất kể ai tính ở đâu hay khi nào.
- Tính kháng đụng độ (collision resistance) — không khả thi để tìm hai đầu vào khác nhau cho cùng một đầu ra.
- Tính một chiều — không thể tái tạo đầu vào nếu chỉ biết đầu ra.
- Tính khuếch tán — một thay đổi nhỏ ở đầu vào tạo ra đầu ra hoàn toàn khác (avalanche effect).

Bốn tính chất này là nền tảng cho việc đóng dấu nội dung trong contract

ContentRegistry của đề tài: nếu một bài đăng bị chỉnh sửa dù chỉ một ký tự, hash mới sẽ khác hash đã lưu on-chain, và việc verify sẽ thất bại ngay lập tức.

2.2.3 Smart Contract

Smart contract là chương trình được lưu và thực thi trên blockchain. Khái niệm này được Nick Szabo đề xuất năm 1994, nhưng chỉ trở thành hiện thực với sự ra đời của Ethereum năm 2015 — nền tảng đầu tiên cung cấp một máy ảo Turing complete cho phép viết logic phức tạp. Một smart contract, khi được triển khai, có một địa chỉ riêng

và có thể được tương tác bởi bất kỳ ai gửi giao dịch tới nó. Mã nguồn của contract sau khi triển khai là bất biến — không thể sửa, chỉ có thể "hủy" bằng cách dùng tương tác và triển khai một contract mới (mọi nỗ lực sửa logic phải qua các pattern phức tạp như proxy contract).

Đặc điểm bất biến này vừa là điểm mạnh (người dùng không cần lo bị thay đổi nội dung giữa chừng) vừa là thách thức (mọi lỗi đều khó vá). Vì lý do này, kiểm thử kỹ trước khi deploy là một yêu cầu bắt buộc trong phát triển smart contract.

2.3 Ethereum và Sepolia testnet

2.3.1 Máy ảo Ethereum (EVM)

Ethereum Virtual Machine là máy ảo dạng stack-based được thiết kế để thực thi mã bytecode của smart contract trên mọi node Ethereum một cách xác định và nhất quán [4]. EVM xử lý lệnh ở mức opcode (mã máy), trong đó mỗi opcode có một chi phí gas cố định phản ánh tài nguyên tính toán hoặc lưu trữ mà nó tiêu thụ.

Trên Ethereum tồn tại hai loại tài khoản, được phân biệt rõ ràng:

- Externally Owned Account (EOA) — tài khoản được kiểm soát bằng một cặp khóa công khai / khóa riêng tư (private key). EOA có thể chủ động tạo và ký giao dịch. Mỗi ví MetaMask của người dùng là một EOA.
- Contract Account — tài khoản chứa bytecode của một smart contract. Contract account không thể tự khởi tạo giao dịch mà chỉ phản ứng khi được EOA hoặc contract khác gọi.

2.3.2 Vòng đời giao dịch

Một giao dịch Ethereum đi qua các giai đoạn sau:

1. Tạo và ký. Người dùng tạo giao dịch với các trường: địa chỉ đích, giá trị ETH gửi, dữ liệu (data payload, thường là lời gọi hàm contract), giới hạn gas, và giá gas. Giao dịch được ký bằng private key của EOA gửi.
2. Phát tán (broadcast). Giao dịch được gửi đến một node Ethereum qua RPC; node lan tỏa giao dịch đến các peer khác theo giao thức P2P, cuối cùng giao dịch nằm trong mempool chờ được đưa vào block.
3. Đào (mine / propose). Các validator (sau khi Ethereum chuyển sang Proof of Stake năm 2022) chọn các giao dịch từ mempool — thường ưu tiên giao dịch trả phí cao hơn — và đóng gói thành một block mới.
4. Xác nhận. Sau khi block được phát hành, mạng tiếp tục đồng thuận các block sau. Mỗi block sau làm tăng độ "chắc chắn" của block cũ — một giao dịch

thường được coi là an toàn sau 1 xác nhận (cho ứng dụng nhỏ), 12 xác nhận (cho thanh toán), hoặc 32 xác nhận (cho giá trị lớn — finality).

2.3.3 Gas – Cơ chế phí

Gas là đơn vị đo lường tài nguyên tính toán trên Ethereum. Mỗi opcode EVM có một chi phí gas cố định: ví dụ phép cộng số nguyên tốn 3 gas, ghi vào storage tốn 20.000 gas (hoặc 5.000 nếu ghi đè giá trị đã có), gửi ETH ra ngoài tốn 9.000 gas. Tổng phí một giao dịch tính theo công thức:

$$\text{Phí (ETH)} = \text{Gas đã dùng} \times \text{Giá gas (ETH / gas)}$$

Người gửi giao dịch phải đặt giới hạn gas (gas limit) — số gas tối đa chấp nhận tiêu thụ — và giá gas (gas price, đơn vị thường là gwei, 1 gwei = 10^{-9} ETH). Nếu gas thực sự dùng vượt giới hạn, giao dịch bị revert nhưng vẫn mất phí — do đó việc chọn gas limit phù hợp là quan trọng. Cơ chế này có hai mục đích: (a) giới hạn để tránh tấn công DoS bằng vòng lặp vô hạn, và (b) tạo ra thị trường định giá cho không gian block.

2.3.4 Mạng thử nghiệm Sepolia

Đề tài sử dụng Sepolia — một mạng thử nghiệm chính thức của Ethereum được giới thiệu năm 2022. ETH trên Sepolia không có giá trị tiền thật, được phân phối miễn phí qua các "faucet" để các nhà phát triển có thể kiểm thử hợp đồng thông minh trong môi trường gần giống mainnet mà không tốn chi phí thực. Các đặc điểm chính của Sepolia:

- Sử dụng cùng cơ chế consensus Proof of Stake với Ethereum mainnet.
- Block time trung bình ~12 giây.
- Etherscan có giao diện riêng tại sepolia.etherscan.io cho phép tra cứu mọi giao dịch, contract đã verify, và event log.
- Là testnet được khuyến nghị bởi Ethereum Foundation sau khi Goerli testnet trước đó bị deprecate cuối năm 2023.

Đề tài chọn Sepolia thay vì Hardhat local (chạy trong RAM của máy phát triển) để có một môi trường gần gũi với production và cho phép kiểm tra trực tiếp trên Etherscan công cộng.

2.4 Solidity và OpenZeppelin

2.4.1 Ngôn ngữ Solidity

Solidity là ngôn ngữ lập trình bậc cao được thiết kế chuyên cho việc viết smart contract trên EVM [5]. Cú pháp của Solidity vay mượn từ JavaScript, C++ và Python, nhưng đã được tinh chỉnh để phù hợp với mô hình thực thi đặc thù của blockchain (xác định, có chi phí gas).

Một số khái niệm cốt lõi của Solidity:

- Contract — đơn vị triển khai cơ bản, tương đương class trong các ngôn ngữ hướng đối tượng. Một contract có thể kế thừa từ một hoặc nhiều contract khác.
- State variable — biến lưu trên storage của contract, tồn tại vĩnh viễn (cho đến khi bị thay đổi). Ghi vào state đắt hơn nhiều so với tính toán trong bộ nhớ tạm.
- Function visibility — bốn mức: public (gọi được từ bên ngoài và bên trong), external (chỉ từ bên ngoài), internal (chỉ trong contract và các contract kế thừa), private (chỉ trong contract).
- Modifier — đoạn mã chèn trước hoặc sau hàm để kiểm tra điều kiện, tương tự decorator. Ví dụ nonReentrant hoặc onlyRole(ADMIN_ROLE).
- Event — phương tiện ghi log có cấu trúc, được lưu trong block log và có thể được index/lắng nghe ở off-chain. Event không chiếm storage, chi phí thấp.
- Custom error (từ Solidity 0.8.4) — cách khai báo lỗi tiết kiệm gas hơn so với revert kèm chuỗi.

Đề tài sử dụng Solidity phiên bản 0.8.24, là phiên bản ổn định gần đây nhất tại thời điểm phát triển. Solidity từ phiên bản 0.8.0 trở đi mặc định kiểm tra tràn số (integer overflow / underflow) ở mọi phép toán số nguyên — một cải tiến quan trọng so với các phiên bản trước cần phải import thư viện SafeMath thủ công.

2.4.2 Thư viện OpenZeppelin

OpenZeppelin Contracts là thư viện smart contract mã nguồn mở được kiểm thử và audited rộng rãi nhất trong hệ sinh thái Ethereum [6]. Thư viện cung cấp các "building block" cho các pattern phổ biến: token chuẩn ERC-20 / ERC-721, kiểm soát truy cập, chống reentrancy, proxy upgrade, và nhiều hơn nữa. Việc sử dụng OpenZeppelin thay vì tự viết các thành phần này từ đầu là một best practice cứng trong cộng đồng Solidity, vì các contract của OpenZeppelin đã trải qua nhiều vòng audit chuyên nghiệp và kiểm thử trong sản phẩm thực với hàng tỷ USD tài sản.

Đề tài sử dụng hai contract của OpenZeppelin trong Charity.sol:

AccessControl — triển khai mô hình Role-Based Access Control on-chain. Mỗi vai trò (role) được biểu diễn bằng một giá trị bytes32, thường được tạo bằng cách băm tên vai trò: keccak256("OPERATOR_ROLE"). Một địa chỉ có thể được cấp (grantRole) hoặc thu hồi (revokeRole) một vai trò bởi tài khoản có vai trò admin.

Hàm hasRole(role, account) trả về true nếu địa chỉ đang giữ vai trò đó. Thông qua modifier onlyRole(role), các hàm có thể được giới hạn chỉ cho phép các tài khoản có vai trò chỉ định gọi.

ReentrancyGuard — cung cấp modifier nonReentrant ngăn một hàm bị gọi đệ quy trong cùng một chuỗi giao dịch. Triển khai sử dụng một biến trạng thái _status được set thành _ENTERED khi hàm bắt đầu và phục hồi thành _NOT_ENTERED sau khi hàm kết thúc. Khi nỗ lực gọi đệ quy, _status đã là _ENTERED và lời gọi bị revert.

Đoạn mã sau minh họa cách kết hợp hai contract này trong Charity.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract Charity is AccessControl, ReentrancyGuard {
    bytes32 public constant OPERATOR_ROLE = keccak256("OPERATOR_ROLE");

    constructor(address admin, address operator) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(OPERATOR_ROLE, operator);
    }

    function unlockMilestone(uint256 id, uint256 idx)
        external
        onlyRole(OPERATOR_ROLE)
        nonReentrant
    {
        // ... logic giải ngân
    }
}
```

Hình 1 Mã nguồn minh họa AccessControl và ReentrancyGuard trong Charity.sol

2.5 Thư viện Ethers.js

Ethers.js là thư viện JavaScript được sử dụng phổ biến nhất để giao tiếp với Ethereum từ phía client (cả frontend trình duyệt và backend Node.js) [7]. Đề tài sử dụng phiên bản 6 — phiên bản viết lại hoàn toàn so với v5, có cải tiến lớn về tính module hóa và sử dụng BigInt gốc của JavaScript thay vì kiểu BigNumber riêng.

Hai khái niệm trung tâm trong Ethers.js là Provider và Signer:

Provider chỉ thực hiện các thao tác *đọc* trên blockchain: gọi hàm view, lấy số dư, lấy receipt giao dịch, lắng nghe event. Provider không cần private key. Đề tài sử

dùng ethers.JsonRpcProvider ở backend và ethers.BrowserProvider (window.ethereum) ở frontend (lấy provider từ MetaMask).

Signer mở rộng Provider, thêm khả năng *ghi* — tạo, ký và gửi giao dịch. Signer phía backend được khởi tạo từ private key trong biến môi trường (new ethers.Wallet(BE_WALLET_PRIVATE_KEY, provider)). Signer phía frontend được lấy từ MetaMask sau khi người dùng đồng ý kết nối.

Để gọi hàm của một smart contract, lập trình viên cần ABI (Application Binary Interface) — đặc tả định dạng của các hàm và event. Ethers v6 hỗ trợ hai dạng ABI: JSON artifact (sinh ra bởi compiler) và Human-Readable ABI — một mảng chuỗi mô tả từng hàm theo cú pháp giống Solidity. Đề tài chọn dạng human-readable trong các service backend để giảm phụ thuộc vào đường dẫn tới file artifact:

2.6 Bảo mật smart contract

2.6.1 Tấn công Reentrancy

Reentrancy là loại lỗ hổng nổi tiếng nhất trong lịch sử Ethereum, gây ra vụ tấn công The DAO năm 2016 — kẻ tấn công rút được khoảng 50 triệu USD ETH ở thời điểm đó [8]. Cơ chế tấn công như sau:

1. Contract nạn nhân có một hàm rút tiền: nó gửi ETH ra ngoài *trước* khi cập nhật state nội bộ (set số dư của người gửi về 0).
2. Khi ETH được gửi tới một địa chỉ contract, hàm receive() của contract đó được gọi tự động.
3. Kẻ tấn công viết một contract trong đó hàm receive() gọi lại hàm rút tiền của nạn nhân.
4. Vì state chưa được cập nhật, kiểm tra số dư vẫn pass, và contract nạn nhân lại gửi ETH lần nữa — quá trình lặp lại cho đến khi cạn ngân quỹ.

2.6.2 Mô hình CEI (Checks-Effects-Interactions)

Phương pháp phòng vệ chuẩn cho reentrancy là Checks-Effects-Interactions [9] — sắp xếp logic theo ba bước theo đúng thứ tự:

1. Checks — xác minh đầu vào và điều kiện (require).
2. Effects — cập nhật state nội bộ.
3. Interactions — chỉ sau cùng mới gọi ra ngoài (transfer ETH, call contract khác).

Khi state được cập nhật trước khi gửi ETH, các nỗ lực gọi đệ quy sẽ thất bại ở bước Checks vì state đã thay đổi. Pattern CEI là biện pháp phòng vệ "miễn phí" (không tốn gas) và là cách làm được khuyến nghị mặc định.

Bổ sung cho CEI, đề tài còn áp dụng ReentrancyGuard của OpenZeppelin để có lớp phòng vệ thứ hai. Các hàm như claimRefund và unlockMilestone trong Charity.sol được trang bị modifier nonReentrant:

```
function claimRefund(uint256 id) external nonReentrant {
    Campaign storage c = campaigns[id];
    require(c.status == Status.FAILED, "Not failed");

    // Effects - cập nhật state TRƯỚC khi gửi ETH ra ngoài
    uint256 amount = contributions[id][msg.sender];
    require(amount > 0, "Nothing to refund");
    contributions[id][msg.sender] = 0;

    // Interactions - gọi ra ngoài sau cùng
    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok, "Transfer failed");

    emit RefundClaimed(id, msg.sender, amount);
}
```

Hình 2 Minh họa áp dụng mô hình CEI (Checks-Effects-Interactions)

2.6.3 Pull payment so với Push payment

Push payment là khi contract chủ động đẩy tiền cho nhiều người nhận trong một giao dịch (thường bằng vòng lặp for). Mô hình này có hai vấn đề: (a) một người nhận có hàm receive() revert sẽ chặn toàn bộ vòng lặp, khóa tiền cho mọi người khác — đây là một dạng tấn công Denial of Service; và (b) gas tiêu thụ tăng tuyến tính theo số người nhận, dễ vượt giới hạn gas của block.

Pull payment đảo ngược trách nhiệm: contract chỉ cập nhật số dư cho mỗi người dùng, và mỗi người dùng tự gọi một hàm withdraw() (hoặc claimRefund()) để nhận tiền của mình. Mô hình này tránh được cả hai vấn đề trên: lỗi của một người không ảnh hưởng người khác, và chi phí gas được phân tán giữa các người nhận.

Đề tài áp dụng pull payment cho Charity.claimRefund — khi một chiến dịch thất bại, mỗi nhà tài trợ tự gọi claimRefund(campaignId) để nhận lại số tiền họ đã đóng, không phụ thuộc vào tổ chức.

2.6.4 Một số rủi ro khác

Integer overflow / underflow. Trong các phiên bản Solidity trước 0.8.0, phép cộng vượt giới hạn uint256 sẽ trả về kết quả wrap (về 0) mà không có cảnh báo nào — kẻ tấn công có thể lợi dụng để tạo các giá trị bất ngờ. Solidity 0.8.0 trở đi mặc định revert

khi tràn số, loại bỏ hoàn toàn lớp lỗi này. Đề tài dùng 0.8.24 nên không phải xử lý thủ công.

Kiểm soát truy cập không đúng. Tất cả hàm public của contract đều có thể được gọi bởi bất kỳ ai trừ khi có biện pháp giới hạn. Một lỗi phổ biến là quên đặt modifier `onlyOwner` hoặc `onlyRole` cho các hàm nhạy cảm. Pattern phòng vệ là Principle of Least Privilege — chỉ cấp đúng quyền cần thiết, và mọi hàm thay đổi state phải được xem xét quyền truy cập một cách rõ ràng.

Tấn công từ chối dịch vụ (DoS). Hai dạng phổ biến là (a) gas bomb — gửi giao dịch kèm hàm `receive()` tiêu thụ nhiều gas khiến lời gọi tới nó hết gas và revert; và (b) block stuffing — kẻ tấn công gửi nhiều giao dịch giá gas cao để chiếm hết không gian block, ngăn các giao dịch hợp lệ được mine. Phòng vệ bằng cách giới hạn độ dài vòng lặp, dùng pull payment, và đặt deadline đủ rộng cho các thao tác phụ thuộc thời gian.

2.7 Công nghệ phụ trợ

2.7.1 JSON Web Token (JWT)

JWT là chuẩn token xác thực không trạng thái, được sử dụng phổ biến trong các API REST [10]. Một JWT gồm ba phần ngăn cách bằng dấu chấm:

`<header>.<payload>.<signature>`

Trong đó *header* khai báo thuật toán ký, *payload* chứa các claim như ID người dùng và thời gian hết hạn, và *signature* được tạo bằng cách ký nội dung header và payload với một khóa bí mật. Khi server nhận token, nó tính lại signature và so sánh — nếu khớp, token hợp lệ và payload đáng tin. Vì server không cần lưu trạng thái, JWT phù hợp với kiến trúc stateless và scale ngang bằng nhiều process. Đề tài lưu JWT có thời gian sống 30 ngày trong `localStorage` của trình duyệt và đính kèm vào header Authorization: Bearer `<token>` qua axios interceptor.

2.7.2 Cloudinary

Cloudinary là dịch vụ Content Delivery Network chuyên cho phương tiện (ảnh, video). Đặc điểm quan trọng được khai thác trong đề tài là khả năng transformation tức thời ở mức URL: chèn các tham số như `f_auto,q_auto,w_400,c_limit` ngay sau `/upload/` trong URL ảnh, Cloudinary sẽ tự động tạo phiên bản ảnh tối ưu (định dạng phù hợp với browser, độ phân giải đúng nhu cầu, chất lượng nén thông minh) và cache lại cho các lần sau. Kỹ thuật là một trong các yếu tố cốt lõi giúp giảm tổng dung lượng tải xuống của trang /charity từ 3.18 MB xuống dưới 800 KB.

2.7.3 Socket.io

Socket.io là thư viện realtime hai chiều cho Node.js. Khác với WebSocket "thuần", Socket.io tự động fallback sang HTTP long polling khi WebSocket không khả dụng (firewall, proxy không hỗ trợ), cung cấp các khái niệm cao cấp như namespace (phân vùng kết nối theo chủ đề) và room (nhóm các socket để broadcast). Đề tài sử dụng Socket.io cho chat realtime, thông báo, gọi thoại signaling, và đồng bộ trạng thái giao dịch on-chain.

2.7.4 Firebase Authentication

Firebase Authentication là dịch vụ xác thực người dùng của Google, hỗ trợ nhiều phương thức (email/password, Google, Facebook, Phone, Anonymous). Đề tài chỉ sử dụng phương thức Google Sign-In: phía frontend, người dùng đăng nhập qua popup của Google và nhận được một ID token (JWT do Google ký); phía backend, ID token được verify qua thư viện firebase-admin để xác minh đó là token thật của Google và chưa hết hạn. Sau khi verify, đề tài tự sinh JWT nội bộ riêng và trả về client (không dùng ID token Google trực tiếp cho các request tiếp theo).

2.7.5 Mongoose

Mongoose là ODM (Object Document Mapper) phổ biến nhất cho MongoDB trong môi trường Node.js. Mongoose bổ sung lớp schema có cấu trúc lên trên MongoDB phi cấu trúc, cung cấp các tính năng: validation tự động khi ghi (required, min, max, enum, match), middleware kiểu pre/post hook (ví dụ pre-save hash password bằng bcrypt), virtual property tính toán, và populate (join nhẹ giữa các collection). Một điểm cần lưu ý là hàm findByIdAndUpdate không trigger pre-save hook — đề tài có quy ước rõ trong tài liệu kiến trúc rằng các thao tác cần hook (đặc biệt là hash password) phải dùng .save() thay vì findByIdAndUpdate.

2.7.6 AWS

Amazon Web Services (AWS) là nền tảng điện toán đám mây cung cấp nhiều dịch vụ hạ tầng như máy chủ ảo, lưu trữ, mạng và bảo mật. Trong phạm vi đề tài, nhóm sử dụng AWS EC2 để triển khai backend Node.js/Express trên môi trường production. Việc triển khai backend trên EC2 giúp hệ thống có một máy chủ riêng để xử lý API, Socket.io, cron job và các tác vụ đồng bộ dữ liệu với blockchain.

Máy chủ EC2 được cấu hình với hệ điều hành Ubuntu, sử dụng PM2 để quản lý tiến trình Node.js và Nginx làm reverse proxy để chuyển tiếp request từ tên miền về backend. Cách triển khai này giúp backend có thể chạy ổn định, tự khởi động lại khi gặp lỗi và dễ dàng mở rộng cấu hình khi hệ thống cần nhiều tài nguyên hơn.

Chương 3 Phân tích và thiết kế hệ thống

3.1 Phân tích yêu cầu

3.1.1 Yêu cầu chức năng

Hệ thống SocialApp được phân thành chín nhóm chức năng chính, bao quát đầy đủ trải nghiệm của một mạng xã hội hiện đại có tích hợp tính năng quyền góp blockchain. Bảng dưới liệt kê các use case chính trong mỗi nhóm.

Bảng 4 Các nhóm chức năng và use case chính

Nhóm	Use case chính
Auth	Đăng ký bằng email/password, đăng nhập email, đăng nhập Google OAuth, đăng nhập bằng ví Web3 (MetaMask), quên mật khẩu, xác thực email, liên kết / hủy liên kết ví
Social	Đăng / sửa / xoá bài đăng (kèm ảnh, video, mention), bình luận lồng nhau (nested comment), like bài và like bình luận, kết bạn (gửi / chấp nhận / hủy), theo dõi (follow / unfollow), tìm kiếm người dùng và bài đăng
Realtime	Chat 1-1 mã hóa end-to-end, chat nhóm, gọi thoại 1-1 qua WebRTC, thông báo realtime (like, comment, mention, kết bạn), đánh dấu đã đọc
Group	Tạo nhóm, mời thành viên, vai trò admin / member trong nhóm, đăng bài trong nhóm, rời nhóm
Story	Đăng story dạng ảnh / video, tự xoá sau 24 giờ (TTL), xem danh sách người đã xem
Save	Bookmark bài đăng, xem danh sách bài đã lưu
Organization	Nộp đơn xin xác minh tổ chức (kèm ảnh chứng minh), quản lý thông tin tổ chức, xem trang công khai của tổ chức theo slug
Charity	Xem danh sách chiến dịch (lọc theo trạng thái và danh mục), xem chi tiết chiến dịch, tạo chiến dịch mới (yêu cầu org đã verified),

Nhóm	Use case chính
	quyên góp ETH, yêu cầu hoàn tiền khi chiến dịch thất bại, kiểm tra dữ liệu on-chain
Admin	Duyệt đơn xin xác minh tổ chức, quản lý người dùng, duyệt báo cáo cột mốc của chiến dịch, ép trạng thái thất bại cho chiến dịch khi có tranh chấp, xử lý báo cáo vi phạm cộng đồng

Tổng cộng có khoảng 25 use case chính trải đều trên 9 nhóm. Đặc thù của đề tài là toàn bộ chức năng blockchain (đăng nhập ví, quyên góp, hoàn tiền, kiểm tra on-chain) đều tùy chọn — người dùng không có ví metamask vẫn có thể sử dụng đầy đủ các tính năng còn lại.

3.1.2 Yêu cầu phi chức năng

Bảng dưới đặc tả các yêu cầu phi chức năng theo bốn nhóm tiêu chí phổ biến: hiệu năng, bảo mật, khả mở rộng, và bảo trì.

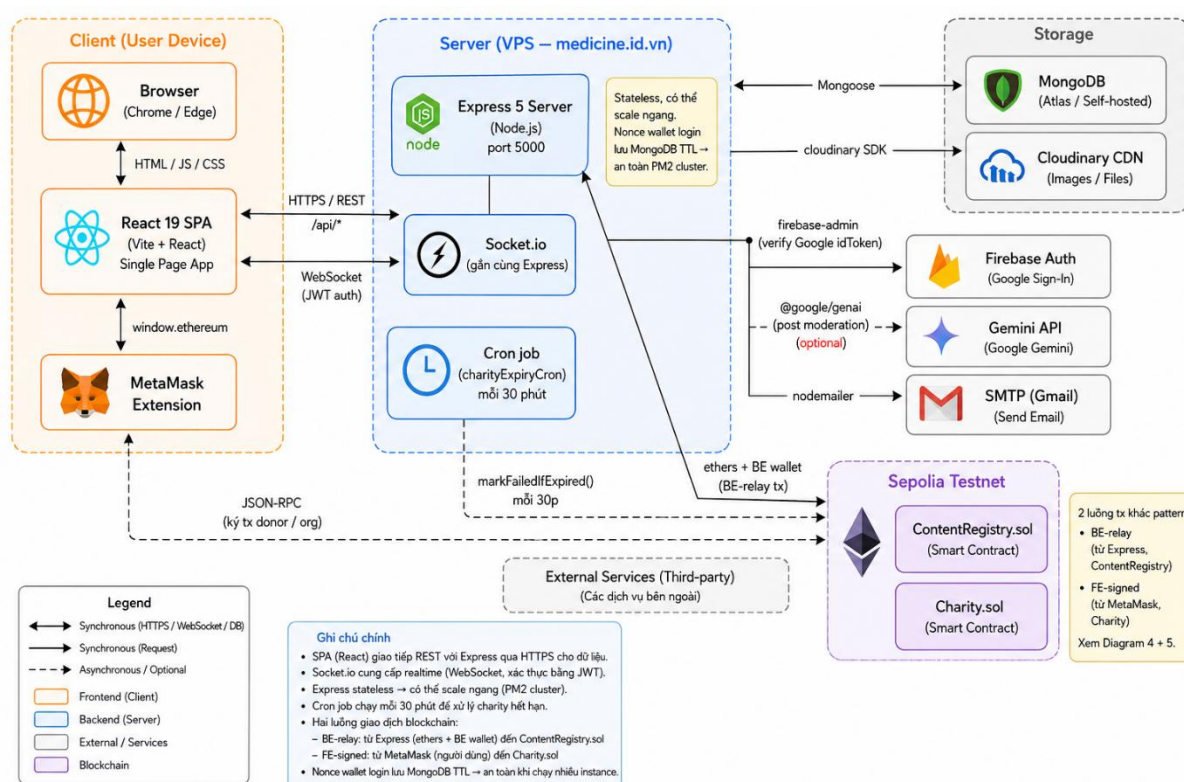
Bảng 5 Yêu cầu phi chức năng

Tiêu chí	Đặc tả cụ thể
Hiệu năng	Trang chủ và danh sách chiến dịch đạt Lighthouse Performance ≥ 75 trên thiết bị di động và ≥ 90 trên desktop. LCP $< 2.5s$ cho desktop; chấp nhận $< 5s$ cho mobile do hạn chế kiến trúc SPA
Bảo mật	Mật khẩu được hash bằng bcrypt với salt round ≥ 10 . Mọi endpoint nhạy cảm có rate-limit. Token JWT có thời gian sống cố định và được verify ở mọi request. Smart contract dùng các thư viện đã audited của OpenZeppelin. Defense in depth ba lớp cho các quy tắc kinh doanh quan trọng
Khả mở rộng	Backend stateless, có thể chạy ở chế độ PM2 cluster với nhiều instance đồng thời mà không cần sticky session. Mọi nonce, session-like data đều lưu MongoDB với TTL thay vì in-memory để an toàn khi scale. CSDL có index cho mọi truy vấn nóng.
Khả dụng	Tính năng blockchain optional — chưa có ví không ảnh hưởng đến tính năng MXH cốt lõi. UI xử lý gracefully khi MetaMask không tồn tại, người dùng đổi tài khoản, hoặc đổi sai mạng (chỉ cho phép Sepolia).

Tiêu chí	Đặc tả cụ thể
Tương thích	Hỗ trợ Chrome, Edge, Firefox phiên bản mới nhất. Responsive cho màn hình mobile ($\geq 360\text{px}$) đến desktop ($\leq 1920\text{px}$).
Bảo trì	Kiến trúc phân tầng nghiêm ngặt giúp thay đổi một tầng không lan sang tầng khác. Hệ thống có CLAUDE.md (tài liệu hướng dẫn AI agent), 4 README (gốc + 3 phần con), 6 sơ đồ kiến trúc và sequence. Quy ước code thống nhất, có constants tập trung và logger chuẩn.

3.2 Kiến trúc tổng thể

Kiến trúc tổng thể của SocialApp được trình bày trong hình, bao gồm bốn cụm thành phần chính cùng các giao thức truyền thông giữa chúng.



Hình 3 Sơ đồ kiến trúc tổng thể của SocialApp

Cụm Client (Thiết bị người dùng). Bao gồm trình duyệt chạy ứng dụng React 19 dưới dạng Single-Page Application và extension MetaMask cho phép tương tác với blockchain. SPA giao tiếp với máy chủ qua hai kênh: HTTP / REST cho các thao tác CRUD thông thường, và WebSocket (Socket.io) cho các cập nhật realtime (tin nhắn mới, thông báo, đồng bộ trạng thái giao dịch). MetaMask kết nối trực tiếp đến mạng Sepolia qua giao thức JSON-RPC để ký các giao dịch của người dùng (donor, org).

Cụm Server. Một quy trình Node.js chạy Express 5 ở cổng 5000, đồng thời mount Socket.io trên cùng cổng đó. Một cron job nội bộ chạy mỗi 30 phút (charityExpiryCron) tự động chuyển trạng thái các chiến dịch đã quá hạn nhưng chưa đủ mục tiêu sang FAILED, để nhà tài trợ có thể yêu cầu hoàn tiền. Server không có trạng thái cục bộ (stateless), cho phép chạy nhiều instance song song qua PM2 cluster.

Cụm Storage. Cơ sở dữ liệu chính là MongoDB Atlas (hosted free tier), giao tiếp qua thư viện Mongoose. Các tài nguyên đa phương tiện (ảnh profile, ảnh bài đăng, ảnh story, cover chiến dịch) được upload qua API backend rồi forward sang Cloudinary CDN để phân phối toàn cầu — tách storage media khỏi database giúp giảm tải MongoDB và cho phép áp dụng các kỹ thuật transformation URL.

Cụm Sepolia Testnet. Bao gồm hai smart contract đã được triển khai và verify trên Etherscan: ContentRegistry (đóng dấu bài đăng) và Charity (quyên góp). Server tương tác với hai contract này qua thư viện Ethers.js: với ContentRegistry, server ký giao dịch bằng ví của chính mình (BE-relay); với Charity, server chỉ đọc và verify, các giao dịch ghi do donor và org tự ký từ MetaMask (FE-signed). Cron job nói trên cũng gọi thẳng vào contract để chuyển state chiến dịch hết hạn.

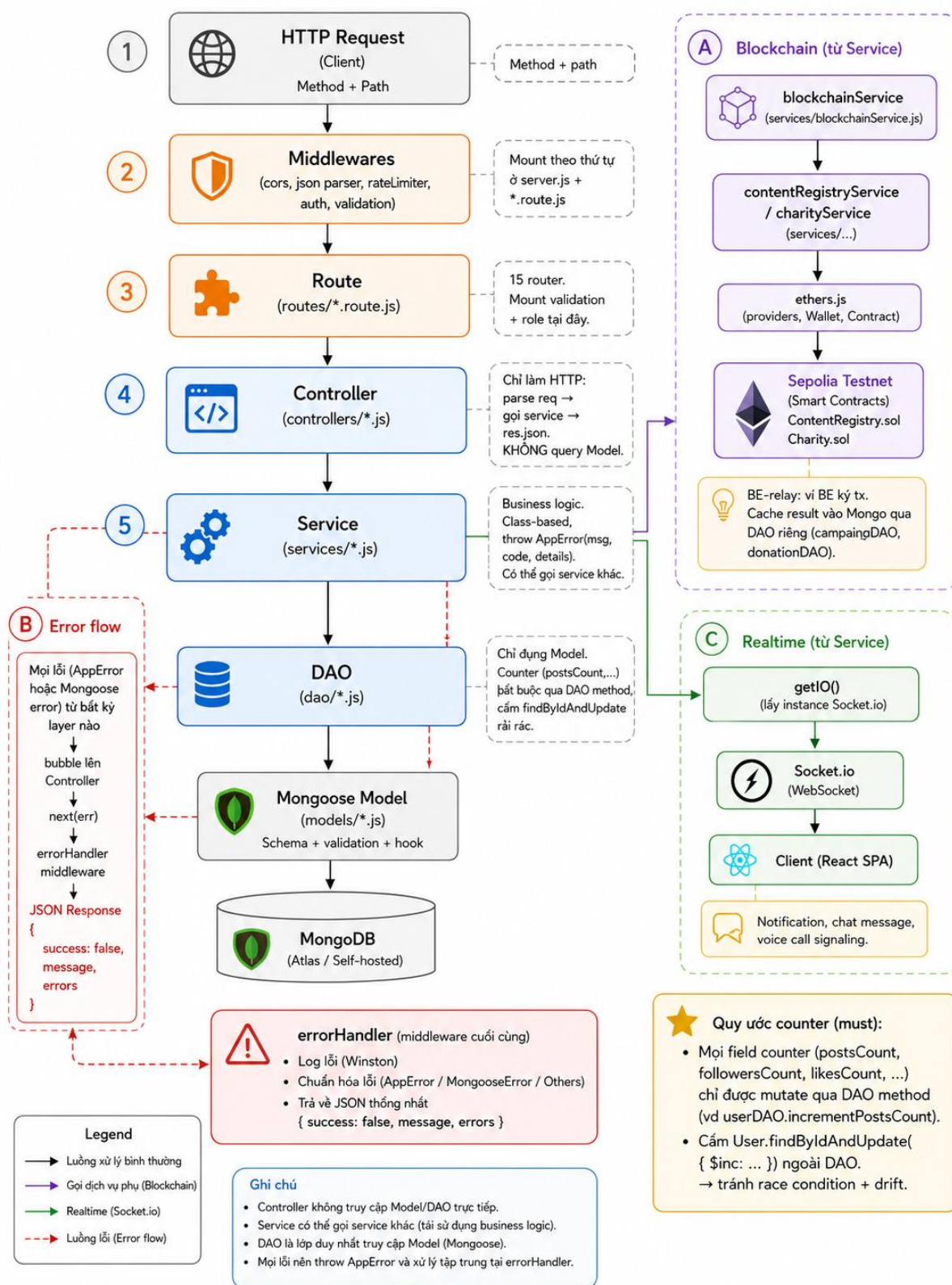
Các dịch vụ bên thứ ba. Server còn tích hợp Firebase Authentication cho việc xác minh ID token Google trong luồng đăng nhập OAuth, Gemini API cho việc kiểm duyệt nội dung bài đăng tự động, và SMTP (Gmail) cho việc gửi email xác thực và khôi phục mật khẩu. Các dịch vụ này đều là tùy chọn — thiếu chúng các tính năng tương ứng sẽ tắt nhưng phần còn lại của hệ thống vẫn hoạt động bình thường.

Một quan sát quan trọng từ sơ đồ là sự tồn tại đồng thời của hai luồng giao dịch khác pattern đến cùng mạng Sepolia: một mũi tên đi từ Server → Sepolia (BE-relay cho ContentRegistry) và một mũi tên đi từ MetaMask → Sepolia (FE-signed cho Charity). Sự khác biệt này không phải tùy tiện mà phản ánh đúng yêu cầu kỹ thuật của hai contract.

3.3 Thiết kế backend

3.3.1 Kiến trúc phân tầng Controller – Service – DAO

Backend của SocialApp được tổ chức theo kiến trúc phân tầng nghiêm ngặt, gồm bảy lớp từ HTTP request đến MongoDB. Hình 4 trình bày sơ đồ chi tiết với hai nhánh phụ cho tích hợp blockchain và xử lý lỗi.



Hình 4 Kiến trúc phân tầng Controller – Service – DAO của backend

Trách nhiệm cụ thể của mỗi tầng được mô tả trong bảng dưới.

Bảng 6 Trách nhiệm các tầng kiến trúc Backend

Tầng	Trách nhiệm	Quy ước cứng
Middleware	CORS, parse JSON, rate-limit, xác thực JWT, validate input	Mount theo thứ tự, validation throw <code>AppError(400, errors)</code>
Route (routes/ *.route.js)	Mount đường dẫn với HTTP verb, gắn middleware cho từng route	15 router theo domain (auth, post, friend, ...)
Controller	Parse req, gọi service, trả res.json. Bắt error → <code>next(err)</code>	KHÔNG query Model trực tiếp; KHÔNG gọi DAO trực tiếp
Service	Logic nghiệp vụ. Class-based, throw <code>AppError(message, code, details)</code>	Có thể gọi service khác cùng tầng; KHÔNG gọi Model
DAO (Data Access Object)	Truy cập Mongoose Model duy nhất. Tên hàm chuẩn: <code>findById</code> , <code>findOne</code> , <code>findMany</code> , <code>create</code> , <code>updateById</code> , <code>softDeleteById</code> , <code>count</code>	Counter (postsCount, followersCount, ...) BẮT BUỘC qua DAO method, không cho phép Model <code>findByIdAndUpdate</code> rải rác
Mongoose Model	Định nghĩa schema, validation, hook (pre-save, post-save)	Soft delete: dùng <code>field deleted: true</code> , <code>deletedAt: Date</code> , không xóa vật lý
MongoDB	Lưu trữ vật lý	Index B-tree cho mọi truy vấn nóng; TTL index cho story 24h và nonce 5 phút

Lý do của kiến trúc phân tầng này là khả năng kiểm thử và bảo trì:

- Service có thể được unit test độc lập vì chỉ gọi DAO — DAO được mock bằng Jest mock dễ dàng.
- Controller mỏng, gần như chỉ làm "ánh xạ" giữa HTTP và service — thay đổi giao thức truyền (như REST sang GraphQL) chỉ ảnh hưởng controller, không ảnh hưởng logic.
- DAO tập trung mọi truy cập database, dễ thay đổi khi cần migrate giữa các database hoặc thêm cache layer.

Một quy ước quan trọng được áp dụng đồng nhất là mọi error đều được throw lên ở tầng phát hiện (service hoặc DAO), không bao giờ được swallow. Service throw class `AppError` với `statusCode` rõ ràng (`new AppError("User not found", 404)`); controller chỉ cần `try/catch` → `next(err)`; middleware errorHandler cuối cùng nhận tất cả error và format thành JSON response thống nhất `{ success: false, message, errors }`. Cách tiếp cận này tránh phân tán mã xử lý lỗi ra nhiều vị trí khác nhau và đảm bảo response shape nhất quán cho client.

Hai nhánh phụ thoát khỏi luồng chính được vẽ trong hình:

- Nhánh Blockchain. Service có thể gọi `blockchainService` (lớp wrap `Ethers.js` provider/signer) để tương tác với smart contract. Mỗi contract có service và DAO cache riêng (`charityService` + `campaignDAO`, `contentRegistryService` không cần DAO riêng vì cache nhúng trong `Post.onChain.*`). Pattern off-chain cache này được trình bày chi tiết ở chương 4 mục 4.5.3.
- Nhánh Realtime. Service có thể emit event qua hàm `getIO()` lấy `Socket.io` server toàn cục, broadcast đến phòng phù hợp (vd `user:userId` cho thông báo, `chat:groupId` cho tin nhắn nhóm).

3.3.2 Authentication và authorization

`SocialApp` hỗ trợ ba phương thức xác thực khác nhau, đáp ứng nhu cầu của các nhóm người dùng đa dạng:

1. Email + mật khẩu. Phương thức cổ điển nhất. Khi đăng ký, mật khẩu được hash bằng `bcrypt` với salt round 10 trước khi lưu Mongo. Khi đăng nhập, mật khẩu nhập vào được so sánh bằng `bcrypt.compare` với hash đã lưu (constant-time comparison chống timing attack).
2. Google OAuth. Người dùng đăng nhập qua popup Google ở phía frontend, nhận về một ID token (JWT do Google ký). ID token được gửi lên backend, verify bằng thư viện `firebase-admin`. Sau verify thành công, server tự sinh JWT nội bộ và trả về cho client.

3. Đăng nhập bằng ví Web3. Quy trình chi tiết:

- Client yêu cầu một nonce ngẫu nhiên cho địa chỉ ví của mình.
- Server tạo nonce, lưu vào collection Nonce của MongoDB với TTL 5 phút (single-use), trả về client.
- Client yêu cầu MetaMask ký một thông điệp chứa nonce.
- Client gửi (walletAddress, signature) lên backend. Backend khôi phục địa chỉ ký từ signature bằng ethers.verifyMessage, so sánh với địa chỉ gửi lên. Nếu khớp và nonce còn hiệu lực, xoá nonce ngay lập tức (single-use) và sinh JWT.

Việc lưu nonce trong MongoDB thay vì Map in-memory là quyết định thiết kế quan trọng: nó cho phép backend chạy ở chế độ cluster nhiều process (PM2) mà nonce vẫn chia sẻ được giữa các process. TTL index của MongoDB tự động xoá nonce cũ sau 5 phút mà không cần job dọn dẹp thủ công.

JWT sau khi sinh chứa payload { id, username, role, ... }, ký bằng HS256 với secret từ biến môi trường, có thời gian sống 30 ngày. Mọi request tới các endpoint cần xác thực đều phải có header Authorization: Bearer <token>. Middleware authMiddleware verify token và gắn req.user cho controller sử dụng.

Middleware roleMiddleware('admin') giới hạn các route admin chỉ cho phép tài khoản có vai trò admin gọi.

3.3.3 Realtime với socket.io

Tính năng realtime của SocialApp (chat, gọi thoại, thông báo, đồng bộ trạng thái giao dịch on-chain) được triển khai qua Socket.io. Server Socket.io được khởi tạo trong file config/socket.js và mount cùng cổng với HTTP server Express, chia sẻ cùng cơ chế xác thực JWT.

Khi client kết nối, một middleware Socket.io xác minh JWT từ trường auth.token của handshake. Nếu hợp lệ, kết nối được duy trì và client được tự động join vào room user:userId để nhận các sự kiện cá nhân (thông báo, tin nhắn riêng).

Các sự kiện được tổ chức theo domain trong thư mục socket/:

- chatHandlers.js — xử lý các event message:send, message:read, typing:start, typing:stop. Khi gửi tin nhắn vào nhóm, server broadcast đến room chat:groupId.
- callHandlers.js — signaling cho gọi thoại 1-1 qua WebRTC. Server chỉ làm relay cho các SDP offer / answer và ICE candidate giữa hai peer; luồng âm thanh thực tế truyền P2P qua WebRTC.

Phía client có hook tùy chỉnh useSocket() (cung cấp bởi SocketContext) để các component lắng nghe và phát event mà không cần làm việc trực tiếp với socket instance — đây là lớp trừu tượng tốt cho việc test và refactor.

3.4 Thiết kế frontend

Frontend được xây dựng bằng React 19 với toolchain Create React App. Cấu trúc tổng thể tuân theo bốn nguyên tắc:

Nguyên tắc 1 — Folder-per-component / page. Mỗi component tái sử dụng và mỗi page là một thư mục riêng chứa file .jsx cùng file .css liền kề. Cách tổ chức này giúp việc xóa hoặc di chuyển một component trở nên dễ dàng (chỉ cần xóa thư mục), và bảo đảm CSS không ảnh hưởng ra ngoài phạm vi component. Project không sử dụng CSS-in-JS hay CSS Modules để giảm phụ thuộc.

Nguyên tắc 2 — API layer tập trung. Mọi lời gọi HTTP đều đi qua một file service trong thư mục src/api/, ví dụ postService.js, charityService.js. Component KHÔNG được gọi axios trực tiếp. Axios instance ở api/axios.js có hai interceptor quan trọng: (a) tự động gắn header Authorization: Bearer <token> cho mỗi request, lấy token từ localStorage, và (b) tự động redirect về / khi nhận status 401, dọn sạch state đăng nhập.

Nguyên tắc 3 — Quản lý trạng thái đa cấp. Trạng thái toàn cục được quản lý qua React Context với bốn provider lồng nhau:

```
<AuthProvider>           // user, JWT, login/logout
  <SocketProvider>         // socket.io client + sự kiện realtime
    <Web3Provider>         // wallet address, signer, chainId, balance
      <BrowserRouter>
        <Suspense fallback={<Loading />}>
          <Routes>
            ...
          </Routes>
        </Suspense>
      </BrowserRouter>
    </Web3Provider>
  </SocketProvider>
</AuthProvider>
```

Hình 5 Cấu trúc thư mục mã nguồn frontend

Thứ tự lồng cổ ý: AuthProvider ngoài cùng vì các provider khác đều phụ thuộc vào trạng thái đăng nhập. Web3Provider ở trong cùng vì nó độc lập với realtime. Trạng thái local trong từng component được quản lý bằng useState hoặc useReducer. Đề tài không sử dụng Redux hay Zustand vì quy mô vừa của project — Context API kết hợp với hook tùy chỉnh đã đủ cho mọi nhu cầu.

Nguyên tắc 4 — Lazy route + code splitting. Mọi page được import qua `React.lazy()` và bao trong `<Suspense>` cấp ứng dụng:

```
const Home = lazy(() => import("./pages/Home/Home"));  
const CharityDetail = lazy(() => import("./pages/CharityDetail/CharityDetail"));
```

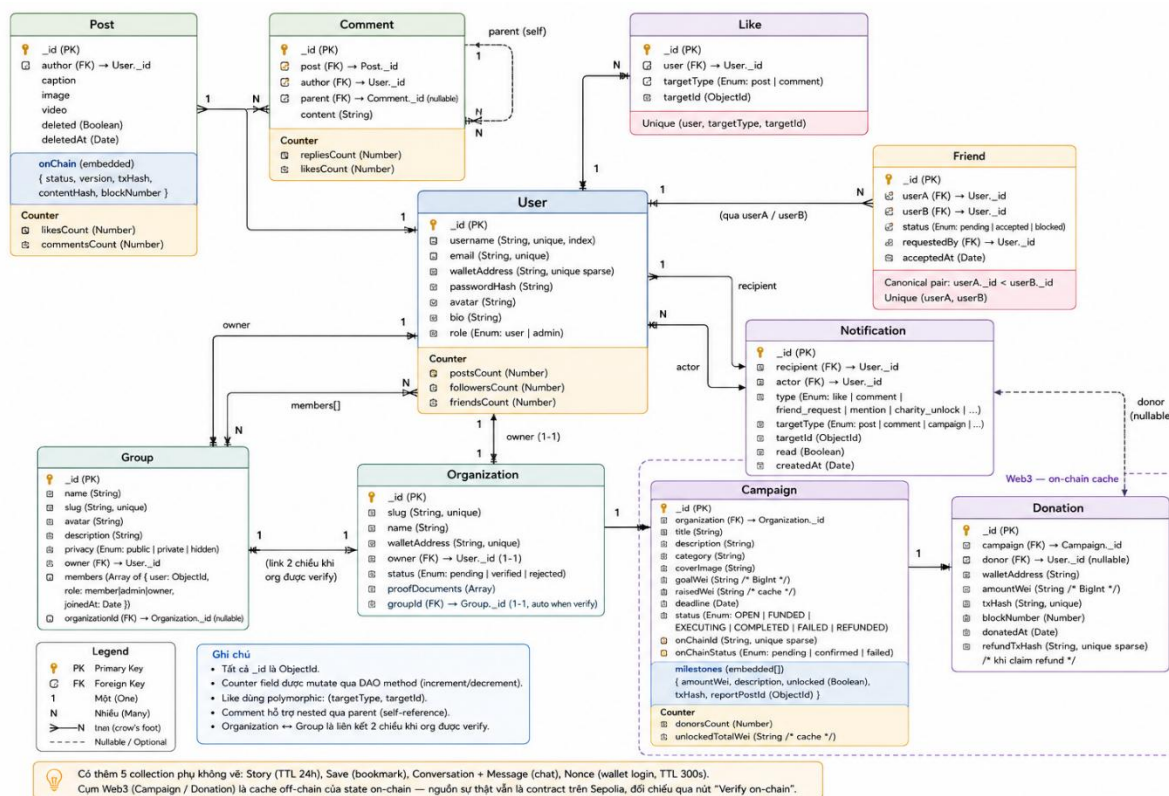
Hình 6 Sơ đồ lồng nhau của các React Context provider

Webpack tự động tách mỗi page thành một chunk JavaScript riêng, chỉ tải khi route được kích hoạt. Bundle khởi tạo do đó chỉ bao gồm các thư viện chung (React, react-router, axios, ethers core) cùng với layout khung — không kéo theo code của các page chưa truy cập. Ngoài lazy ở cấp page, một số component nặng và ít dùng cũng được lazy ở cấp component (ví dụ `DonateModal`, `ClaimRefundModal` — chỉ load khi người dùng bấm "Ủng hộ" / "Yêu cầu hoàn tiền"; xem chi tiết ở chương 5 mục 5.2).

Internationalization. Đề tài hỗ trợ hai ngôn ngữ chính là tiếng Việt (mặc định) và tiếng Anh thông qua thư viện `react-i18next`. Mọi chuỗi hiển thị cho người dùng đều phải gọi qua hàm `t("key")` thay vì `hardcode`, giúp việc dịch hoàn toàn tách rời khỏi code. Các file ngôn ngữ nằm trong thư mục `src/localization/`. Các ngôn ngữ còn lại (tiếng Pháp, Nhật, Đức) vẫn đang được hỗ trợ ở một số trang nhưng vẫn chưa hoàn chỉnh)

3.5 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu MongoDB của SocialApp gồm khoảng 15 collection, trong đó 10 collection cốt lõi được trình bày trong sơ đồ ERD. Năm collection phụ (Story, Save, Conversation, Message, Nonce) được mô tả ngắn ở cuối mục này để giảm độ phức tạp của sơ đồ chính.



Hình 7 Sơ đồ ERD các collection chính của cơ sở dữ liệu

3.5.1 Các thực thể chính

User là thực thể trung tâm, kết nối với hầu hết các thực thể khác. Các trường quan trọng: username (unique, có index để truy vấn nhanh), email (unique), walletAddress (unique sparse — sparse cho phép nhiều user chưa link ví, nhưng đã link thì phải duy nhất), passwordHash (không bao giờ trả về client), role (user hoặc admin), và ba counter postsCount, followersCount, friendsCount được mutate duy nhất qua các DAO method.

Post lưu nội dung bài đăng. Trường embedded onChain là một sub-document ghi nhận trạng thái đóng dấu blockchain với các field: status (pending, registered, failed), version (v2 cho post mới, null cho post legacy), txHash, contentHash, blockNumber. Trường này tồn tại song song với nội dung post — nội dung là source of truth, blockchain chỉ là "minh chứng" có thể kiểm chứng.

Friend lưu quan hệ bạn bè giữa hai người dùng. Một quy ước thiết kế quan trọng là canonical pair: hai user trong một quan hệ bạn bè luôn được lưu theo thứ tự userA._id < userB._id (so sánh theo ObjectId). Quy ước này tránh việc phải insert và truy vấn hai dòng cho cùng một quan hệ, đồng thời tạo điều kiện áp unique index (userA, userB) để chống dupe.

Organization lưu tổ chức từ thiện. Quan hệ một-một với User (chủ tổ chức) và một-một với Group (nhóm cộng đồng được tạo tự động khi tổ chức được verify).

Trường walletAddress unique đảm bảo một ví chỉ gắn với một tổ chức.

Campaign lưu chiến dịch quyên góp. Hai trường goalWei và raisedWei là chuỗi BigInt (vì Number JavaScript không đủ dài biểu diễn số ETH trong wei). Trường embedded milestones[] lưu danh sách cột mốc với cấu trúc { amountWei, description, unlocked, txHash, reportPostId }. Trường onChainStatus (pending, confirmed, failed) phản ánh trạng thái xác nhận on-chain, và onChainId là ID của chiến dịch trên contract Solidity (unique sparse).

Donation lưu một giao dịch quyên góp. Trường txHash unique đảm bảo idempotent — nếu cùng một txHash được record hai lần, hệ thống không tạo hai bản ghi mà trả lại bản ghi cũ. refundTxHash (unique sparse) lưu hash của giao dịch hoàn tiền nếu có.

Notification lưu thông báo cá nhân. Trường target mang tính polymorphic — chứa postId, commentId, campaignId, ... tùy loại thông báo, cùng với targetType để biết cách render.

Like ghi nhận một lượt like. Cũng polymorphic với targetType (post hoặc comment) và targetId. Composite unique (user, targetType, targetId) đảm bảo một user chỉ like một mục đúng một lần.

3.5.2 Các thực thể phụ

Năm collection sau không xuất hiện trong sơ đồ chính để giữ độ rõ:

- Story — bài story 24 giờ, có TTL index trên createdAt để MongoDB tự xóa sau khi hết hạn.
- Save — bookmark, lưu cặp (user, post) với composite unique.
- Conversation + Message — chat 1-1 và chat nhóm. Message.content được mã hóa end-to-end ở client trước khi lưu Mongo (đề tài dùng AES-GCM với khóa phái sinh từ shared secret giữa hai user).
- Nonce — nonce cho luồng đăng nhập ví, TTL 5 phút.

Cụm Web3 (Organization → Campaign → Donation) trong sơ đồ ERD được đóng khung riêng để nhấn mạnh đây là cache off-chain của state on-chain. Nguồn sự thật cuối cùng vẫn là contract trên Sepolia; cache Mongo chỉ phục vụ cho việc đọc nhanh và truy vấn theo các tiêu chí phức tạp (sắp xếp, lọc, full-text). Cơ chế đồng bộ giữa hai bên được trình bày chi tiết ở chương tiếp theo.

3.6 Thiết kế smart contract

Đây là mục trọng tâm của chương — trình bày thiết kế chi tiết hai smart contract của đề tài. Mục 3.6.1 giới thiệu ContentRegistry, mục 3.6.2 giới thiệu Charity (phức tạp hơn nhiều), và mục 3.6.3 tổng kết hai pattern tích hợp blockchain khác biệt mà đề tài đã phân biệt rõ ràng.

3.6.1 ContentRegistry — Đóng dấu Bài đăng

ContentRegistry là smart contract nhỏ gọn (khoảng 40 dòng Solidity) giải quyết vấn đề toàn vẹn nội dung bài đăng đã nêu ở chương 1. Interface gồm hai hàm và một event:

```
contract ContentRegistry {
    struct Post {
        bytes32 contentHash;
        address owner;
        uint256 timestamp;
        bool exists;
    }

    mapping(string => Post) private posts;

    event PostRegistered(string postId, address owner, uint256 timestamp);

    function registerPost(string memory postId, bytes32 contentHash) public {
        require(!posts[postId].exists, "Post ID already exists");
        posts[postId] = Post({
            contentHash: contentHash,
            owner: msg.sender,
            timestamp: block.timestamp,
            exists: true
        });
        emit PostRegistered(postId, msg.sender, block.timestamp);
    }

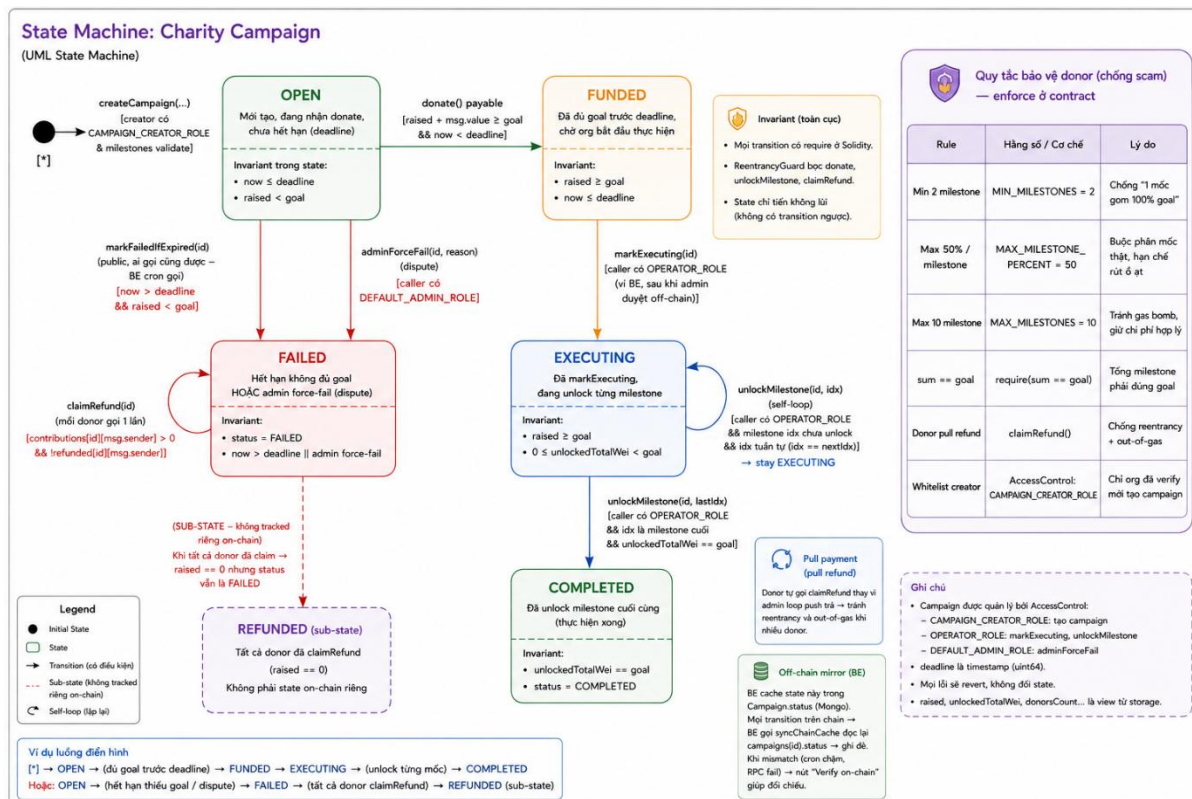
    function verifyPost(string memory postId) public view returns (Post memory) {
        require(posts[postId].exists, "Post does not exist");
        return posts[postId];
    }
}
```

Hình 8 Interface và lưu trữ của smart contract ContentRegistry

Đặc điểm thiết kế quan trọng là contract không hiểu định dạng của content hash — kiểu bytes32 chỉ là 32 byte opaque. Toàn bộ logic về cách tính hash, version v1 / v2, và cách verify nằm hoàn toàn ở off-chain (trong contentRegistryService của backend). Điều này có hai lợi ích: contract đơn giản đến mức không cần audit phức tạp, và mọi sự thay đổi về chiến lược hash trong tương lai không yêu cầu redeploy contract.

3.6.2 Charity — Quyên góp theo Cột mốc

Charity là smart contract chính của đề tài, dài khoảng 300 dòng Solidity, kế thừa từ AccessControl và ReentrancyGuard của OpenZeppelin. Contract triển khai một state machine sáu trạng thái với các transition được trình bày trong hình sau:



Hình 9 Sáu trạng thái của Charity Campaign

Bảng 7 Sáu trạng thái của Charity Campaign

Trạng thái	Ý nghĩa
OPEN	Mới tạo, đang nhận quyên góp, chưa hết hạn
FUNDED	Đã đủ mục tiêu trước hạn, chờ tổ chức bắt đầu thực hiện
EXECUTING	Tổ chức đã bắt đầu, đang giải ngân từng cột mốc
COMPLETED	Đã giải ngân cột mốc cuối cùng
FAILED	Hết hạn không đủ mục tiêu, hoặc admin force-fail vì tranh chấp
REFUNDED	Tất cả nhà tài trợ đã hoàn tiền (sub-state khái niệm của FAILED)

Các transition đều có **invariant** kiểm tra ở Solidity (require):

- OPEN → FUNDED: tự động khi $\text{raised} + \text{msg.value} \geq \text{goal} \ \&\& \ \text{now} < \text{deadline}$.
- OPEN → FAILED: khi `markFailedIfExpired(id)` được gọi (sau khi $\text{now} > \text{deadline} \ \&\& \ \text{raised} < \text{goal}$) — hàm public, ai cũng gọi được, đề tài có cron backend gọi mỗi 30 phút.
- FUNDED → EXECUTING: khi operator (ví backend) gọi `markExecuting(id)` sau khi xác nhận tổ chức cam kết bắt đầu.
- EXECUTING → EXECUTING (self-loop): mỗi lần operator gọi `unlockMilestone(id, idx)` cho một cột mốc chưa unlock.
- EXECUTING → COMPLETED: tự động khi unlock cột mốc cuối cùng.
- FAILED → FAILED (self-loop): donor gọi `claimRefund(id)` để nhận lại số tiền đã đóng. Hàm này áp dụng pattern pull payment + CEI + ReentrancyGuard như đã trình bày ở chương 2.

Bảng 8 Bốn vai trò của AccessControl

Vai trò	Người giữ	Quyền
DEFAULT_ADMIN_ROLE	Ví admin (đề tài dùng chung ví backend cho gọn)	Whitelist / unwhitelist tổ chức, force-fail chiến dịch khi có tranh chấp
CAMPAIGN_CREATOR_ROLE	Tổ chức đã được admin duyệt off-chain và whitelist on-chain	Gọi <code>createCampaign(...)</code>
OPERATOR_ROLE	Ví backend	<code>markExecuting</code> , <code>unlockMilestone</code> (sau khi admin duyệt báo cáo cột mốc off-chain) — không có quyền rút tiền
Donor (mọi địa chỉ)	—	<code>donate(campaignId)</code> payable, <code>claimRefund(campaignId)</code> khi FAILED

Bảng 9 Quy tắc bảo vệ donor (chống lừa đảo)

Quy tắc	Hàng số contract	Lý do
Tối thiểu 2 cột mốc	MIN_MILESTONES = 2	Chống case "1 cột mốc gom 100% mục tiêu"
Mỗi cột mốc tối đa 50% mục tiêu	MAX_MILESTONE_PERCENT = 50	Buộc phân ít nhất 2 cột mốc thực sự, không lệch
Tối đa 10 cột mốc	MAX_MILESTONES = 10	Tránh gas bomb khi unlock nhiều cột mốc
sum(milestoneAmounts) == goal	Loop require	Tổng cột mốc phải đúng mục tiêu, không thừa thiếu
Pull payment cho refund	—	Donor tự claimRefund, contract không loop push — chống reentrancy + out-of-gas

Các quy tắc này được enforce thêm ở backend service và frontend form như đã đề cập, tạo thành defense in depth ba lớp.

Tách metadata off-chain. Để tiết kiệm chi phí gas, contract chỉ lưu hash 32 byte của metadata (bytes32 metadataHash) cho mỗi chiến dịch — toàn bộ thông tin mô tả (tiêu đề, mô tả chi tiết, ảnh bìa, danh mục) được lưu ở MongoDB. Backend đảm bảo tính nhất quán bằng cách: khi tạo chiến dịch, tính metadataHash = keccak256(JSON.stringify({...})) và gửi cùng với giao dịch tạo; bất kỳ ai sau này cũng có thể tự tính lại hash từ metadata Mongo và so sánh với hash on-chain để verify.

3.6.3 So sánh BE-relay và FE-signed pattern

Đây là một trong những đóng góp chuyên môn của đề tài: phân biệt rõ hai pattern tích hợp blockchain khác nhau, mỗi pattern phù hợp với một loại use case khác nhau.

Bảng 10 So sánh pattern BE-relay và FE-signed

Tiêu chí	BE-relay (ContentRegistry)	FE-signed (Charity)
Người ký giao dịch	Backend (ví hệ thống)	Người dùng (donor / org) qua MetaMask
msg.sender trong contract	Địa chỉ ví backend	Địa chỉ ví người dùng
Ai trả gas	Backend	Người dùng
Người dùng cần có ETH?	Không	Có
Cách gắn attribution	Nhúng authorId trong content hash (off-chain)	msg.sender trực tiếp (on-chain)
Trải nghiệm người dùng	Liên mạch, không cần ví ETH cho thao tác miễn phí	Yêu cầu ví + ETH + ký giao dịch
Khi nào nên dùng	Thao tác miễn phí, không liên quan tiền của user, contract không enforce msg.sender	Thao tác liên quan ví / tiền của user, hoặc contract enforce định danh msg.sender
Số API endpoint cần	1 endpoint (BE tự gọi contract)	2 endpoint: /prepare (chuẩn bị payload) + /record (verify event sau khi user đã ký)

Tại sao Charity bắt buộc FE-signed: khi tạo chiến dịch, contract Charity ghi `campaigns[id].beneficiary = msg.sender` — địa chỉ này sẽ là người duy nhất nhận tiền giải ngân khi unlock cột mốc. Nếu backend relay, msg.sender sẽ là ví backend, dẫn đến sai bản chất (tiền sẽ vào ví backend chứ không phải tổ chức). Tương tự với donate, contract ghi `contributions[id][msg.sender] += msg.value` — nếu backend

relay mọi donation, tất cả sẽ được gắn nhãn là của ví backend, không thể hoàn tiền đúng người sau này được. Vì vậy hai hàm này bắt buộc phải do người dùng tự ký.

Tại sao ContentRegistry chọn BE-relay: nếu yêu cầu mỗi người dùng tự trả gas khi đăng bài, hầu hết người dùng sẽ không sử dụng tính năng này (Sepolia tuy miễn phí nhưng phải đi xin faucet, mainnet thì tốn phí thật). Trải nghiệm đăng bài liền mạch là yêu cầu cứng. Vì ContentRegistry không có ràng buộc nào về msg.sender (contract không quan tâm ai gọi), pattern BE-relay là lựa chọn tự nhiên — và attribution được giải quyết ở off-chain qua hash version 2 như đã trình bày ở mục 3.6.1.

Quy tắc lựa chọn pattern. Khi thêm contract mới trong tương lai, đề tài đặt ra quy tắc: mặc định FE-signed nếu contract enforce msg.sender (giải ngân, ghi nhận chủ sở hữu, ...); BE-relay nếu không có ràng buộc trên và muốn UX liền mạch. Nguyên tắc đơn giản này giúp tránh các lỗi thiết kế phổ biến trong các đề tài blockchain ban đầu — nơi nhà phát triển thường chọn BE-relay theo quán tính (vì dễ triển khai hơn) mà không nhận ra pattern này không tương thích với ngữ nghĩa của contract.

3.7 Bảo mật smart contract

Triết lý bảo mật xuyên suốt SocialApp là defense in depth — không tin tuyệt đối vào bất kỳ tầng nào, mọi quy tắc kinh doanh quan trọng đều được kiểm tra ở ba tầng độc lập: smart contract Solidity, backend service, và frontend form. Cách tiếp cận này đảm bảo hệ thống vẫn an toàn ngay cả khi một tầng bị bypass hoặc có lỗi.

Bảng 11 Defense in depth cho các quy tắc kinh doanh quan trọng

Quy tắc	Tầng Solidity	Tầng Backend	Tầng Frontend
Min 2 cột mốc / chiến dịch	require(milestones.length >= MIN_MILESTONES)	_validateAndParseCampaignPayload throw AppError(400)	Form validate() báo lỗi khi nhập 1 cột mốc
Max 50% / cột mốc	require(amount <= goal * MAX_MILESTONE_PERCENT / 100)	Loop check trong service	Live warning khi user gõ số vượt 50%

Quy tắc	Tầng Solidity	Tầng Backend	Tầng Frontend
Tổng cột mốc = mục tiêu	require(sum == goal)	Tính lại sum và so sánh	Hiển thị badge "tổng đúng / sai" realtime
Chỉ verified org tạo campaign	onlyRole(CAMPAIGN_CREATOR_ROLE)	Service kiểm tra org.status === "verified"	Trang /charity/create ẩn form nếu org chưa verify
Idempotent record donation	—	txHash unique constraint Mongo	—

Lý do phải lặp lại quy tắc ở ba tầng:

- Tầng frontend cung cấp UX tốt — báo lỗi ngay khi người dùng gõ thay vì để gửi lên rồi mới báo. Tuy nhiên, vì code chạy ở client, dễ bị bypass (mở DevTools, sửa code, gọi API trực tiếp).
- Tầng backend chống các lời gọi API cố ý vượt ngoài UI. Tầng này cũng thường cung cấp thông báo lỗi chi tiết hơn (ví dụ "Số dư của bạn không đủ" thay vì chỉ "Transaction reverted"). Tuy nhiên backend cũng có thể bị compromise — nếu attacker chiếm được server, họ có thể tự gọi contract bằng ví hệ thống.
- Tầng smart contract là tầng cuối cùng — không thể bị bypass vì code đã immutable on-chain. Ngay cả khi attacker chiếm cả backend lẫn frontend, contract vẫn từ chối các giao dịch sai cấu trúc.

Chương 4 Cài đặt và triển khai

4.1 Môi trường phát triển

Đề tài được phát triển trên Windows 11 với các phiên bản công cụ được chốt cố định trong package.json để bảo đảm khả năng tái lập. Bảng dưới liệt kê các công cụ, phiên bản và vai trò của từng công cụ trong dự án.

Bảng 12 Các công cụ và phiên bản sử dụng

Nhóm	Công cụ	Phiên bản	Vai trò
Runtime	Node.js	22.21.1 LTS	Thực thi backend Node và toolchain frontend
	npm	11.10.1	Quản lý phụ thuộc cho cả ba thư mục
Backend	Express	5.1	Web framework chính, mount Socket.io cùng cổng
	Mongoose	8.19.2	ODM cho MongoDB
	Socket.io	4.8.1	Realtime bidirectional
	Ethers.js	6.16.0	Tương tác Ethereum, wrap provider/signer
	bcrypt	3.0.2	Hash mật khẩu
	jsonwebtoken	9.0.2	Sinh và verify JWT
	node-cron	4.2.1	Lập lịch cron job nội bộ
	firebase-admin	13.6.0	Verify ID token Google OAuth
	cloundinary	2.8.0	SDK upload media
Frontend	React	19.2.0	Thư viện UI
	react-router-dom	7.12.0	Routing client-side

Nhóm	Công cụ	Phiên bản	Vai trò
	axios	1.12.2	HTTP client
	i18next	25.7.4	Đa ngôn ngữ vi/en
	ethers (FE)	6.16.0	Ký giao dịch FE-signed qua MetaMask
	react-hot-toast	2.6.0	Hệ thống toast
Blockchain	Hardhat	2.22.0	Compile, test, deploy smart contract
	Solidity	0.8.24	Ngôn ngữ smart contract
	OpenZeppelin Contracts	5.6.1	Thư viện audited (AccessControl, ReentrancyGuard)
Hạ tầng	MongoDB Atlas	Free M0	Database hosted
	Cloudinary	Free	CDN media
	Sepolia Testnet	—	Mạng thử nghiệm Ethereum
	Alchemy / Infura	—	Node RPC để gọi Sepolia
	Ubuntu 22.04	—	OS trên AWS EC2

Cấu trúc monorepo. Toàn bộ mã nguồn nằm trong một repository duy nhất với ba thư mục con song song: backend/, frontend/, và blockchain/. Mỗi thư mục có package.json riêng và được build / chạy độc lập. Lựa chọn monorepo (thay vì ba repo tách rời) giúp việc đồng bộ thay đổi liên quan đến cả ba phần (ví dụ: thêm field on-chain mới phải đồng thời sửa contract Solidity, ABI ở backend, và state ở frontend) trở nên gọn trong một commit, dễ rà lại lịch sử.

4.2 Cài đặt backend

4.2.1 Cấu trúc thư mục và tổ chức module

Backend được tổ chức theo đúng kiến trúc bảy tầng đã thiết kế ở chương 3. Mỗi tầng tương ứng với một thư mục con, không trộn vai trò. Cấu trúc tóm tắt:

```

backend/
├── server.js           # entry, mount routes /api/*, init socket, cron
├── config/            # database, cloudinary, firebase, socket, email
├── routes/            # *.route.js, 15 router theo domain
├── controllers/       # HTTP layer, gọi service
├── services/          # business logic, throw AppError
├── dao/               # truy cập Mongoose Model
├── models/            # Mongoose schemas
├── middlewares/       # auth, role, errorHandler, rateLimiter, validation
├── utils/             # AppError, logger, emailService, ...
├── helpers/           # postHelper (formatPostsWithMetadata), generate
├── socket/            # chatHandlers, callHandlers
├── jobs/              # charityExpiryCron
├── constants/         # pagination limits, validation limits, enum
└── scripts/           # script migration, batch (chạy thủ công)

```

Hình 10 Cấu trúc thư mục backend

Tại thời điểm hoàn thành đề tài, backend có 15 router, 15 service (13 service nghiệp vụ + 2 service blockchain wrapper), 14 DAO và hơn 20 Mongoose model. Mọi router đều được mount cùng prefix /api/ trong server.js. Việc gom prefix chung cho phép tách riêng apiLimiter rate-limit chỉ áp dụng cho /api/ mà không ảnh hưởng các route hệ thống khác.

Bảng 13 Danh sách các api backend

Domain	Router	Service	DAO
Auth	auth.route.js	authService	userDAO
User	user.route.js	userService	userDAO, followDAO
Post	post.route.js	postService + contentRegistryService	postDAO, userDAO, likeDAO
Comment	comment.route.js	commentService	commentDAO, postDAO, likeDAO

Domain	Router	Service	DAO
Friend	friend.route.js	friendService	friendDAO, userDAO
Notification	notification.route.js	notificationService	notificationDAO
Chat	chat.route.js	chatService	conversationDAO, messageDAO
Group	group.route.js	groupService	groupDAO, userDAO
Story	story.route.js	storyService	storyDAO
Save	save.route.js	saveService	saveDAO
Admin	admin.route.js	adminService	nhiều DAO
Upload	upload.route.js	uploadService	– (Clouinary SDK)
Web3	web3.route.js	web3Service + content RegistryService	userDAO, postDAO
Organization	organization.route.js	organizationService + charityService.whitelistOrg	organizationDAO
Charity	charity.route.js	charityService	campaignDAO, donationDAO, organizationDAO

4.2.2 Một module tiêu biểu — Charity

Module Charity là module phức tạp nhất, đại diện đầy đủ cho cả ba tầng và tương tác với blockchain. File router charity.route.js chỉ làm việc mount đường dẫn cùng middleware:

```
// charity.route.js – trích đoạn
router.get("/campaigns", listCampaignsValidation, charityController.listCampaigns);
router.get("/campaigns/mine", authMiddleware, charityController.listMyCampaigns);
router.get("/campaigns/:id", mongoIdValidation, charityController.getCampaignDetail);

// FE-signed: 2 endpoint cho luồng tạo chiến dịch
router.post("/campaigns/prepare", authMiddleware, createCampaignValidation,
  charityController.prepareCampaign);
router.post("/campaigns/record", authMiddleware, recordCampaignValidation,
  charityController.recordCampaign);

// Admin only
router.post("/campaigns/:id/execute", authMiddleware, requireRole("admin"),
  mongoIdValidation, charityController.markExecuting);
router.post("/admin/force-fail/:id", authMiddleware, requireRole("admin"),
  mongoIdValidation, charityController.adminForceFail);
```

Hình 11 Cấu trúc file router charity.route.js

Controller mỏng, làm đúng vai trò "router HTTP":

```
// charityController.js – mẫu một handler
exports.recordDonation = async (req, res, next) => {
  try {
    const userId = req.user?.id; // có thể null nếu user chưa link wallet
    const data = await charityService.recordDonation(userId, {
      onChainCampaignId: Number(req.body.onChainCampaignId),
      txHash: req.body.txHash,
    });
    res.status(201).json({ success: true, data });
  } catch (err) {
    next(err);
  }
};
```

Hình 12 Controller mỏng đóng vai trò router HTTP

Toàn bộ business logic nằm trong charityService.js, khoảng 900 dòng với 15 phương thức. Phần đặc biệt quan trọng là validation chống lừa đảo được hardcode trong `_validateAndParseCampaignPayload` — enforce đủ năm quy tắc bảo vệ donor. Trích đoạn cốt lõi thực hiện kiểm tra "tổng cột mốc bằng mục tiêu" và "mỗi cột mốc tối đa 50%":

```
// services/charityService.js - trích _validateAndParseCampaignPayload
const maxPercentN = BigInt(MAX_MILESTONE_PERCENT); // 50
let sum = 0n;

for (const [i, m] of milestones.entries()) {
  const mWei = ethers.parseEther(String(m.amountEth));

  // mWei * 100 <= goalWei * MAX_PERCENT ⇔ amount/goal <= 50%
  // Sắp xếp lại tránh phép chia BigInt vì BigInt không có decimal.
  if (mWei * 100n > goalWei * maxPercentN) {
    const actualPercent = Number((mWei * 100000n) / goalWei) / 100;
    throw new AppError(
      `Milestone ${i + 1} exceeds ${MAX_MILESTONE_PERCENT}% of goal ` +
      `(${actualPercent.toFixed(1)}%)`,
      400
    );
  }
  sum += mWei;
}

if (sum !== goalWei) {
  throw new AppError("Sum of milestones must equal goal", 400);
}
```

Hình 13 Validation chống lừa đảo trong charityService.js

Đây là một ví dụ tiêu biểu cho việc xử lý số ETH ở backend. Vì đơn vị nhỏ nhất wei lớn hơn dài Number của JavaScript ($\text{Number.MAX_SAFE_INTEGER} \approx 9 \times 10^{15}$, trong khi $1 \text{ ETH} = 10^{18} \text{ wei}$), mọi phép tính số ETH ở backend đều phải dùng BigInt. Ràng buộc này lan ra cả schema Mongo (goalWei, raisedWei đều lưu kiểu String để giữ nguyên chữ số) và sang frontend.

4.2.3 Quy ước throw lỗi — AppError và errorHandler

Mọi service throw lỗi bằng class AppError thay vì Error chuẩn:

```
// utils/AppError.js
class AppError extends Error {
  constructor(message, statusCode, errors = null) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true; // phân biệt với bug crash
    if (errors) this.errors = errors; // field-level errors
  }
}
```

Hình 14 Class AppError thay cho Error chuẩn

Middleware errorHandler cuối chuỗi mount tại server.js phân nhánh dựa trên cờ isOperational

```
// middleware/errorHandler.js
module.exports = (err, req, res, next) => {
  if (err.isOperational) {
    // Lỗi có thể dự đoán – trả response chuẩn
    return res.status(err.statusCode).json({
      success: false,
      message: err.message,
      ...(err.errors && { errors: err.errors }),
    });
  }
  // Lỗi không mong đợi – Log + trả 500 generic
  logger.error("UNEXPECTED ERROR:", err);
  res.status(500).json({
    success: false,
    message: "Something went wrong",
    error: process.env.NODE_ENV === "development" ? err.message : undefined,
  });
};
```

Hình 15 Middleware errorHandler phân nhánh theo cờ isOperational

Cờ isOperational là cách phân biệt cơ bản nhưng rất hữu hiệu giữa "lỗi nghiệp vụ dự kiến" (sai mật khẩu, hết hạn, không đủ ETH...) và "bug" (lỗi trong code, mất kết nối DB, ...). Lỗi nghiệp vụ luôn được trả về client với message rõ ràng; bug thì chỉ log đầy đủ ở server và trả về client một message generic để tránh leak thông tin nội bộ. Quy ước này khiến controller chỉ cần một mẫu duy nhất:

```
exports.handler = async (req, res, next) => {
  try {
    const data = await someService.doThing(...);
    res.json({ success: true, data });
  } catch (err) {
    next(err); // mọi error chuyển errorHandler – không bao giờ res.status(...) trong controller
  }
};
```

Hình 16 Cron job charityExpiryCron.js

4.2.4 Cron job và xử lý thời gian

Cron job duy nhất của hệ thống là [charityExpiryCron.js](#) — định kỳ rà soát các chiến dịch đã quá hạn nhưng chưa đủ mục tiêu, gọi contract Charity.markFailed(id) để chuyển sang trạng thái FAILED, mở khóa quyền claimRefund cho donor. Cron schedule mặc định */30 * * * * (mỗi 30 phút), có thể override bằng env CHARITY_EXPIRY_CRON cho mục đích demo (* / 2 * * * * để thực thi với chu kỳ 2 phút).

Đoạn code chính của job:

```
// jobs/charityExpiryCron.js
async function runExpiryCheck() {
  if (!process.env.CHARITY_ADDRESS) return; // skip nếu blockchain chưa cấu hình
  if (isRunning) { // guard chống concurrent
    logger.warn("Charity expiry cron: previous run still in progress, skipping");
    return;
  }
  isRunning = true;

  try {
    const expired = await campaignDAO.findMany(
      {
        status: "OPEN",
        onChainStatus: "confirmed",
        deadline: { $lt: new Date() },
      },
      { limit: BATCH_SIZE, lean: true, sort: { deadline: 1 } }
    );

    for (const campaign of expired) {
      try {
        await charityService.markFailedIfExpired(campaign._id);
      } catch (err) {
        // Race condition: donation cuối cùng đẩy raised >= goal vào đúng phút expired
        // → contract throw "already met its goal" – ghi log warn và bỏ qua.
        logger.warn(`Charity expiry: skip _id=${campaign._id} - ${err.message}`);
      }
    }
  } finally {
    isRunning = false;
  }
}
```

Hình 17 Đoạn mã chính của cron job đánh dấu chiến dịch quá hạn

Có ba quyết định thiết kế đáng lưu ý trong job này:

1. Guard isRunning trên cùng process. Một lần chạy có thể mất vài phút vì mỗi markFailed là một giao dịch Sepolia (~15-30 giây block confirm × 20 chiến dịch). Nếu bỏ guard, hai lần chạy chồng nhau sẽ gửi cùng một giao dịch markFailed cho cùng chiến dịch hai lần với cùng nonce → một lần fail. Guard này chỉ chống chồng trong cùng process; nếu chạy PM2 cluster nhiều instance, mỗi instance đều có cron riêng. Đề tài chấp nhận giới hạn này vì markFailed là idempotent trên contract — gọi lại sau khi đã FAILED chỉ revert chứ không gây hậu quả; hơn nữa contract sẽ reject tất cả giao dịch lặp ngoại trừ giao dịch đầu tiên.
2. Initial run sau 15 giây. setTimeout(runExpiryCheck, 15_000) đảm bảo job chạy một lần ngay sau khi server khởi động (catch các chiến dịch đã expired trong lúc server down) — nhưng đợi 15 giây để connectDatabase (không await ở server.js) kịp hoàn tất.
3. Skip khi CHARITY_ADDRESS chưa set. Ở môi trường dev không có blockchain, job tự bỏ qua — không crash app. Đây là pattern xuyên suốt: tính năng blockchain optional ở mọi tầng.

Cron job được khởi động trong server.js chỉ bằng một dòng startCharityExpiryCron() sau khi initializeSocket(server)

4.3 Cài đặt frontend

4.3.1 Cấu trúc thư mục và lazy routing

Frontend giữ đúng nguyên tắc folder-per-component và lazy route đã phác thảo ở chương 3. Với cấu trúc:

```
frontend/src/
├─ App.jsx           # routes (lazy), wrap providers, mount toaster
├─ index.js          # entry, register service worker
├─ api/              # axios instance + 1 service per domain
├─ contexts/         # AuthContext, SocketContext, Web3Context, ThemeContext
├─ pages/            # mỗi page 1 thư mục .jsx + .css
├─ components/       # reusable, folder-per-component
├─ hooks/            # useForm, useAvatarError, useVoiceCall
├─ localization/     # i18n vi/en
├─ utils/            # cloundinary, web3Errors, dataHelpers, toast
└─ constants.js
```

Hình 18 Cấu trúc thư mục mã nguồn frontend

Tổng số trang xấp xỉ 30, tổng số component reusable xấp xỉ 40. Phần lớn component dưới 200 dòng — component lớn nhất là PostCard (gần 500 dòng do gộp logic like, comment-preview, lazy hydrate ảnh, mention).

Trong App.jsx, mọi page đều được lazy:

```
const Home = lazy(() => import("./pages/Home/Home"));
const Charity = lazy(() => import("./pages/Charity/Charity"));
const CharityDetail = lazy(() => import("./pages/CharityDetail/CharityDetail"));
const CreateCampaign = lazy(() => import("./pages/CreateCampaign/CreateCampaign"));
// ... gần 30 page

<AuthProvider>
  <SocketProvider>
    <Web3Provider>
      <BrowserRouter>
        <Suspense fallback={<Loading />}>
          <Routes>
            <Route path="/" element={<PrivateRoute><Home /></PrivateRoute>} />
            <Route path="/charity" element={<Charity />} />
            <Route path="/charity/create" element={<PrivateRoute><CreateCampaign /></PrivateRoute>} />
          </Routes>
        </Suspense>
      </BrowserRouter>
    </Web3Provider>
  </SocketProvider>
</AuthProvider>
```

Hình 19 Khai báo lazy import cho các page trong App.jsx

Webpack 5 (qua CRA) tự tách mỗi `lazy(() => import(...))` thành một chunk riêng — tên file dạng `static/js/4.chunk.js`. Build production sinh ra khoảng 35 chunk JavaScript, trong đó bundle khởi tạo (bao gồm React, react-router, axios, ethers core) chỉ ~290KB gzipped.

4.3.2 Axios instance và xử lý 401

`api/axios.js` là điểm tập trung mọi HTTP request, thực thi đúng "Nguyên tắc — API layer tập trung". File này có hai interceptor:

```
// api/axios.js
const axiosInstance = axios.create({
  baseURL: process.env.REACT_APP_API_URL || "http://localhost:5000/api",
  headers: { "Content-Type": "application/json" },
  timeout: 10000,
});

// Request: tự gắn Authorization header
axiosInstance.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response: 401 + có token → auto Logout (token hết hạn)
axiosInstance.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401 && error.config?.headers?.Authorization) {
      localStorage.removeItem("token");
      localStorage.removeItem("user");
      window.location.href = "/";
    }
    return Promise.reject(error);
  }
);
```

Hình 20 Hai interceptor của Axios instance

Logic `hasAuthHeader` quan trọng: nếu request KHÔNG có Authorization header (ví dụ gọi `/api/auth/login` lúc chưa đăng nhập) mà server trả 401, tuyệt đối không tự động đăng xuất — đó là kết quả mong đợi của login fail, không phải token hết hạn. Phân biệt này tránh được bug "form đăng nhập sai mật khẩu nhưng UI vẫn hoạt động bình thường".

4.3.3 Context Web3 — Quản lý ví và chuyển mạng

`Web3Context.jsx` là Context phức tạp nhất của frontend, quản lý tám thứ cùng lúc: địa chỉ ví, signer ethers, chainId, số dư ETH, trạng thái connect/loading, và ba listener của `window.ethereum` (`accountsChanged`, `chainChanged`, `disconnect`). Context exports các API: `connectWallet()`, `disconnectWallet()`, `switchToSepolia()`, `refreshBalance()`.

Một quy tắc đặt ra ở chương 3 là tính năng blockchain optional — UI phải xử lý gracefully khi MetaMask không tồn tại, khi user đổi tài khoản, hoặc khi user chuyển sai mạng. Web3Context implement cả ba:

```
// Web3Context.jsx - Listener accountsChanged
window.ethereum.on("accountsChanged", async (accounts) => {
  if (accounts.length === 0) {
    // User disconnect từ MetaMask UI
    setWalletAddress(null); setSigner(null); setBalance(null);
    toast(t("web3.disconnected"));
  } else if (accounts[0] !== walletAddress) {
    // User switch sang ví khác - re-init signer
    setWalletAddress(accounts[0]);
    const provider = new ethers.BrowserProvider(window.ethereum);
    setSigner(await provider.getSigner());
    toast(t("web3.account_switched"));
  }
});

// Listener chainChanged
window.ethereum.on("chainChanged", (chainId) => {
  if (chainId !== SEPOLIA_CHAIN_ID) {
    setSigner(null); // signer thuộc network sai → invalidate
    toast.error(t("web3.wrong_network"));
  } else {
    // Trở lại Sepolia - restore signer
    const provider = new ethers.BrowserProvider(window.ethereum);
    provider.getSigner().then(setSigner);
    toast.success(t("web3.network_ok"));
  }
});
```

Hình 21 Xử lý các trạng thái của ví Web3 trong Web3Context

Quy tắc fetch balance: Trong ethers v6, signer.provider thường trả null khi signer được tạo từ BrowserProvider đã bị invalidate (ví dụ sau khi đổi network). Đề tài bám một quy tắc cứng: luôn dùng new ethers.BrowserProvider(window.ethereum) mới khi cần fetch số dư, không bao giờ dùng signer.provider.

4.4 Cài đặt Smart Contract

4.4.1 ContentRegistry.sol

Contract ContentRegistry được giữ tối giản như đã thiết kế — chỉ 38 dòng Solidity hữu ích, không kế thừa thư viện nào, không có modifier. Đây là lựa chọn cố ý: contract càng ít code, attack surface càng nhỏ, audit càng đơn giản. Toàn bộ contract:

```
// blockchain/contracts/ContentRegistry.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract ContentRegistry {
    struct Post {
        bytes32 contentHash;
        address owner;
        uint256 timestamp;
        bool exists;
    }

    mapping(string => Post) private posts;

    event PostRegistered(string postId, address owner, uint256 timestamp);

    function registerPost(string memory postId, bytes32 contentHash) public {
        require(!posts[postId].exists, "Post ID already exists");
        posts[postId] = Post({
            contentHash: contentHash,
            owner: msg.sender,
            timestamp: block.timestamp,
            exists: true
        });
        emit PostRegistered(postId, msg.sender, block.timestamp);
    }

    function verifyPost(string memory postId) public view returns (Post memory) {
        require(posts[postId].exists, "Post does not exist");
        return posts[postId];
    }
}
```

Hình 22 Mã nguồn smart contract *ContentRegistry.sol*

Bảy quan sát thiết kế:

1. bytes32 contentHash — opaque 32 byte, contract không hiểu định dạng; toàn bộ logic version v1 / v2 nằm ở off-chain.
2. mapping(string => Post) — key là postId chuỗi để khớp Mongo ObjectId, không phải uint256 tự sinh.
3. require ngay đầu hàm registerPost chống ghi đè post — vĩnh viễn immutable một khi đã register.
4. event PostRegistered cho phép off-chain index nhanh (qua event filter của RPC).
5. block.timestamp lấy từ block đã mine, không phải từ user input → không thể giả mạo thời điểm đăng.
6. Không có hàm update hay delete — phù hợp ngữ nghĩa "đóng dấu", không phải "lưu trữ".
7. Không có payable, không nhận ETH — contract không bao giờ giữ tiền của ai, đơn giản hóa audit.

4.4.2 Charity.sol

Contract Charity dài khoảng 300 dòng, kế thừa hai thư viện OpenZeppelin:

```
// blockchain/contracts/Charity.sol – đầu file
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract Charity is AccessControl, ReentrancyGuard {
    bytes32 public constant CAMPAIGN_CREATOR_ROLE = keccak256("CAMPAIGN_CREATOR_ROLE");
    bytes32 public constant OPERATOR_ROLE = keccak256("OPERATOR_ROLE");

    enum Status { OPEN, FUNDED, EXECUTING, COMPLETED, FAILED }

    struct Milestone { uint256 amount; bool unlocked; }

    struct Campaign {
        address beneficiary;    // ví org – gán cố định = msg.sender khi createCampaign
        uint256 goal;
        uint256 raised;
        uint256 deadline;
        uint256 unlockedTotal;
        Status status;
        bytes32 metadataHash;    // hash của metadata off-chain (title, desc, ...)
        bool exists;
        Milestone[] milestones;
    }

    uint256 public constant MIN_MILESTONES = 2;
    uint256 public constant MAX_MILESTONES = 10;
    uint256 public constant MAX_MILESTONE_PERCENT = 50;

    mapping(uint256 => Campaign) public campaigns;
    mapping(uint256 => mapping(address => uint256)) public contributions;
    uint256 public nextCampaignId;

    constructor(address admin, address operator) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(OPERATOR_ROLE, operator);
    }
}
```

Hình 23 Khai báo kế thừa của contract Charity.sol

Năm event phục vụ off-chain index và verify ở backend:

```
event CampaignCreated(uint256 indexed id, address indexed beneficiary,
    uint256 goal, uint256 deadline, bytes32 metadataHash);
event Donated(uint256 indexed id, address indexed donor,
    uint256 amount, uint256 newRaised);
event StatusChanged(uint256 indexed id, uint8 from, uint8 to);
event MilestoneUnlocked(uint256 indexed id, uint256 indexed idx, uint256 amount);
event RefundClaimed(uint256 indexed id, address indexed donor, uint256 amount);
```

Hình 24 Năm event phục vụ off-chain index và verify

Backend dựa vào các event này để verify giao dịch. Tham số indexed cho phép filter event theo campaign id và donor address trong một lệnh gọi RPC, tốc độ ~10x so với scan toàn bộ block.

Hàm claimRefund áp dụng đầy đủ pattern Checks-Effects-Interactions

```
function claimRefund(uint256 id) external nonReentrant {
    Campaign storage c = campaigns[id];
    require(c.exists, "Campaign does not exist");
    require(c.status == Status.FAILED, "Campaign not failed");

    // Checks
    uint256 amount = contributions[id][msg.sender];
    require(amount > 0, "Nothing to refund");

    // Effects – zero out TRƯỚC khi transfer (CEI)
    contributions[id][msg.sender] = 0;

    // Interaction – transfer cuối cùng
    (bool ok, ) = msg.sender.call{value: amount}("");
    require(ok, "Refund transfer failed");

    emit RefundClaimed(id, msg.sender, amount);
}
```

Hình 25 Hàm *claimRefund* áp dụng pattern *Checks-Effects-Interactions*

Modifier `nonReentrant` kế thừa từ `ReentrancyGuard` chặn re-entry; việc set `contributions[id][msg.sender] = 0` **trước** call đảm bảo ngay cả nếu attacker tìm được cách bypass `nonReentrant`, lần re-entry thứ hai sẽ thấy `amount = 0` và revert ở `require(amount > 0)`

4.4.3 Triển khai lên Sepolia

Script triển khai `deploy-charity.js` tự động hóa quy trình deploy, gồm balance guard và in lệnh verify.

```
// scripts/deploy-charity.js – phần chính
const [deployer] = await hre.ethers.getSigners();
const balance = await hre.ethers.provider.getBalance(deployer.address);
const minWei = hre.ethers.parseEther("0.05"); // tối thiểu để deploy + verify

if (balance < minWei) {
    throw new Error(`Insufficient balance. Have ${hre.ethers.formatEther(balance)} ETH, ` +
        `need ≥ 0.05 ETH. Nạp faucet trước khi deploy.`);
}

// Constructor: admin + operator (đề tài dùng chung ví BE cho gọn).
const admin = deployer.address;
const operator = deployer.address;

await hre.run("compile");
const Charity = await hre.ethers.getContractFactory("Charity");
const charity = await Charity.deploy(admin, operator);
await charity.waitForDeployment();

const address = await charity.getAddress();
console.log("CHARITY_ADDRESS:", address);
console.log(`Etherscan: https://sepolia.etherscan.io/address/${address}`);
console.log(`Verify: npx hardhat verify --network sepolia ${address} ${admin} ${operator}`);
```

Hình 26 Script triển khai *deploy-charity.js*

Quy trình deploy đầy đủ thực hiện trong đề tài:

1. Cài `hardhat.config.js` với network `sepolia` trỏ tới Alchemy RPC, accounts: `[process.env.BE_WALLET_PRIVATE_KEY]`.
2. Nạp ETH testnet vào ví BE (vài faucet, mỗi lần ~0.5 ETH).

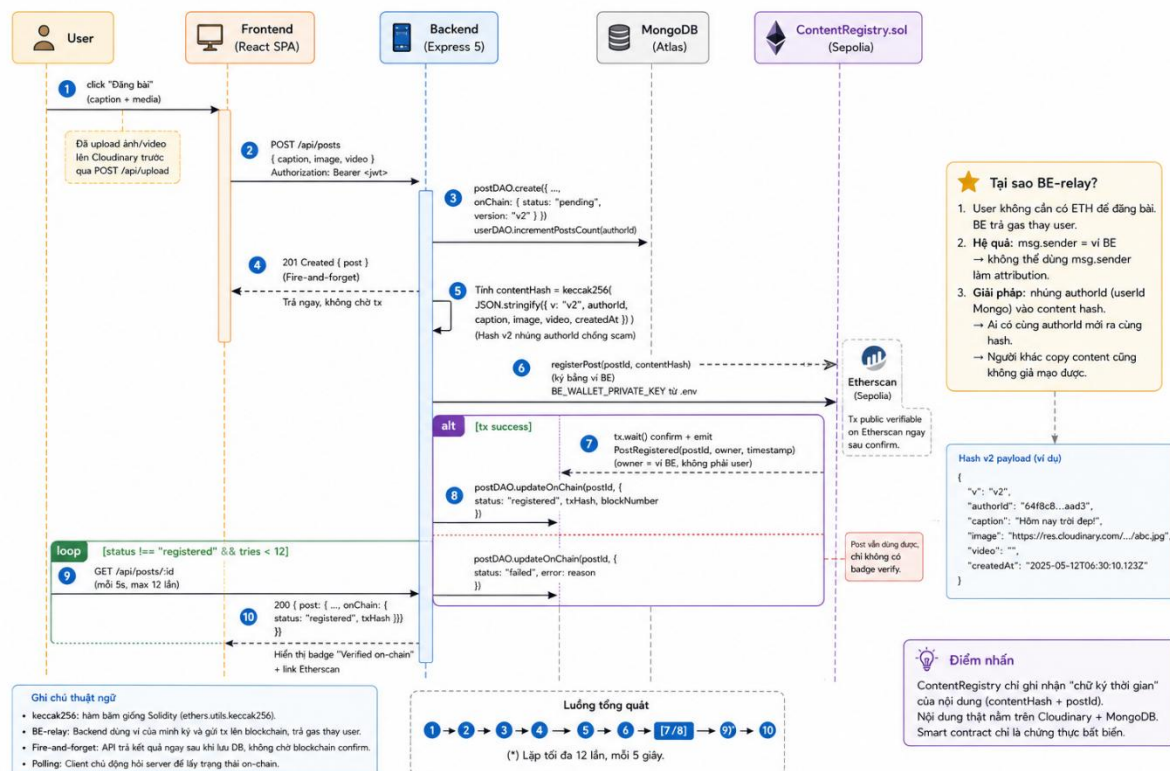
3. Chạy `npx hardhat run scripts/deploy-charity.js --network sepolia` — script tự compile, deploy, in địa chỉ và lệnh verify.
4. Chạy `npx hardhat verify --network sepolia <address> <admin> <operator>` — Etherscan kiểm tra bytecode khớp source và publish source public. Người ngoài có thể đọc trực tiếp source code và gọi function read trên giao diện Etherscan mà không cần dựng client.
5. Copy `CHARITY_ADDRESS` vào `backend/.env` và `REACT_APP_CHARITY_ADDRESS` vào `frontend/.env`.
6. Restart backend (cron job auto-start), test luồng đầy đủ trên `medicine.id.vn`.

Hai địa chỉ contract trên Sepolia (cập nhật cuối tháng 4-2026):

- ContentRegistry: 0x52be...c0E1 (xem bản đầy đủ tại `backend/.env.example`)
- Charity: 0x083e...9Eaa

4.5 Tích hợp blockchain

4.5.1 Flow đăng bài và đóng dấu (ContentRegistry)



Hình 27 Sơ đồ luồng đăng bài và đóng dấu qua ContentRegistry

Luồng này thực hiện đúng pattern fire-and-forget đã đặt ra: API tạo post không đợi blockchain confirm, response trả ngay sau khi lưu MongoDB; backend confirm ngầm ở

background rồi cập nhật `post.onChain.*`; frontend poll mỗi 5 giây để biết khi nào badge "verified" xuất hiện.

Các bước tương tác (đánh số tương ứng với mũi tên trong hình):

1. Người dùng nhập caption + ảnh, bấm "Đăng" trong `CreatePostModal`.
2. Frontend `POST /api/posts (multipart)` — đính kèm ảnh đã upload sẵn lên Cloudinary từ trước.
3. `postController.createPost` parse req → gọi `postService.createPost(userId, payload)`.
4. `postService` lưu Mongo qua `postDAO.create(...)` với `onChain.status = "pending"`.
5. `postService` gọi `userDAO.incrementPostsCount(userId)` (counter mutate qua DAO).
6. `postService` trả response `{ success: true, post }` → controller → client ($\approx 200\text{ms}$).
7. Sau khi response trả về (fire-and-forget), `postService` gọi `contentRegistryService.registerPost(postId, post, authorId)` — không await ở luồng response, chỉ `.catch(err => logger.error(...))`.
8. `contentRegistryService.computeContentHash(post, { version: "v2", authorId })` tính hash keccak256.
9. `contract.registerPost(postId, contentHash)` — signer là ví BE, gas trả từ ví BE.
10. `tx.wait()` — đợi block Sepolia mine ($\sim 15\text{-}30$ giây).
11. `postDAO.updateById(postId, { onChain: { status: "registered", txHash, blockNumber, contentHash, version: "v2" } })`.

Trong khoảng thời gian 5 đến 30 giây từ bước 6 đến bước 11, frontend `PostCard` poll `GET /api/posts/:id` mỗi 5 giây tối đa 12 lần. Khi `post.onChain.status === "registered"`, `VerifiedBadge` xuất hiện cùng link Etherscan.

Trích đoạn fire-and-forget từ `postService.js`:

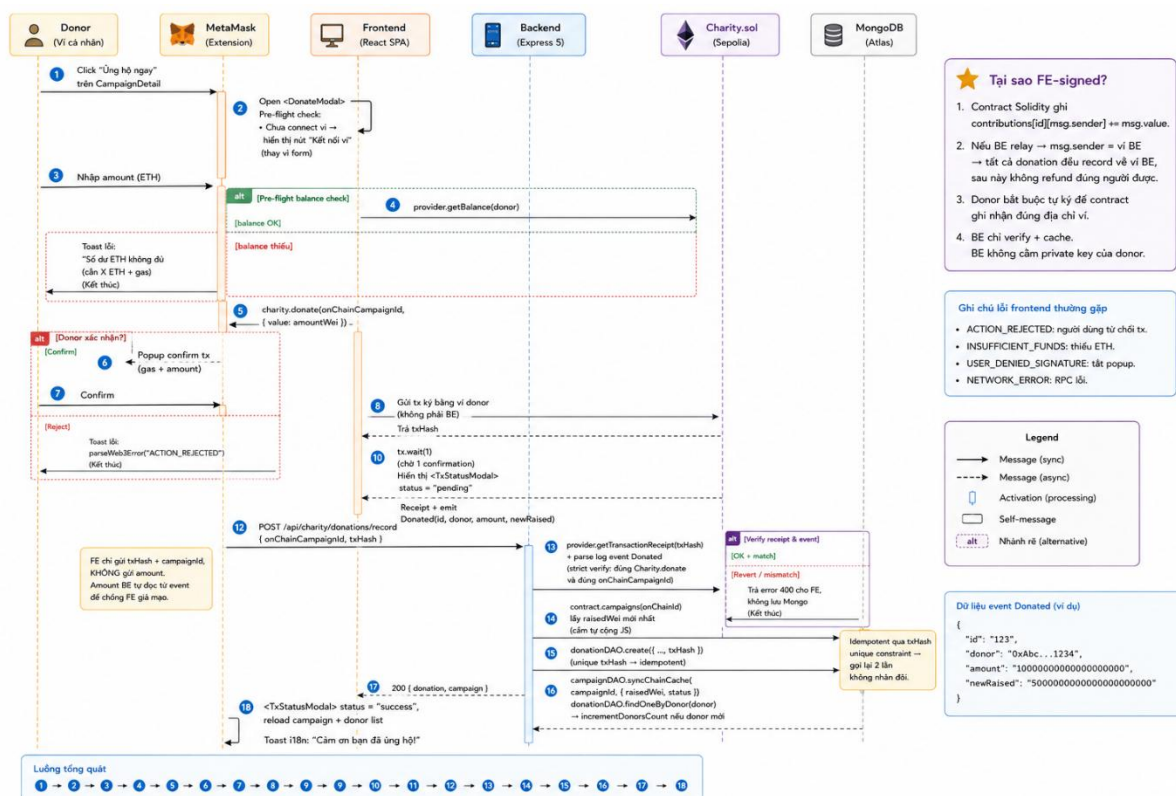
```
// postService.createPost – đoạn cuối
const post = await postDAO.create(payload);
await userDAO.incrementPostsCount(userId);

// Fire-and-forget – không await để response không bị block bởi Sepolia
contentRegistryService
  .registerPost(post._id, post, userId)
  .then(({ txHash, blockNumber, contentHash, version }) =>
    postDAO.updateById(post._id, {
      "onChain.status": "registered",
      "onChain.txHash": txHash,
      "onChain.blockNumber": blockNumber,
      "onChain.contentHash": contentHash,
      "onChain.version": version,
    })
  )
  .catch((err) => {
    logger.error("ContentRegistry: registerPost background failed:", err.message);
    postDAO.updateById(post._id, {
      "onChain.status": "failed",
      "onChain.errorMsg": err.message,
    });
  });
return post;
```

Hình 28 Trích đoạn fire-and-forget trong postService.js

Trường hợp lỗi (gas thấp, RPC drop, contract revert), onChain.status chuyển "failed" và badge hiển thị trạng thái "không xác minh được" thay vì biến mất hoàn toàn — minh bạch với người dùng cuối.

4.5.2 Flow donate và record (Charity)



Hình 29 Sơ đồ luồng quyên góp và ghi nhận qua Charity

Luồng này phức tạp hơn vì giao dịch do người dùng tự ký, backend chỉ làm công việc verify event sau khi tx confirm:

1. Donor mở /charity/:id, bấm "Ủng hộ" → DonateModal hiện ra, show form số ETH.
2. Donor nhập 0.05 ETH, bấm "Xác nhận".
3. DonateModal gọi `assertSepolia()` — kiểm tra chainId.
4. DonateModal đọc balance qua `new ethers.BrowserProvider(window.ethereum)`, kiểm tra `balance >= amount + gasReserve (0.002 ETH)` — pre-flight tránh gửi tx chắc chắn fail.
5. DonateModal gọi `contract.donate(onChainCampaignId, { value: amountWei })` với `signer = await provider.getSigner()`.
6. MetaMask popup hiện thị giao dịch + ước tính gas.
7. Donor bấm "Confirm" — MetaMask ký tx và submit lên Sepolia.
8. `tx.wait(1)` — đợi 1 block confirmation.
9. Sau khi tx confirm, frontend gọi POST `/api/charity/campaigns/:id/donations/record` với body `{ onChainCampaignId, txHash }`.
10. `charityController.recordDonation` → `charityService.recordDonation(userId, payload)`.
11. BE strict verify — không tin số FE gửi:
 - `provider.getTransactionReceipt(txHash)` → check `receipt.status === 1` và `receipt.to === CHARITY_ADDRESS`.
 - Parse log với `iface.getEvent("Donated").topicHash` để lấy event `Donated(id, donor, amount, newRaised)`.
 - Khớp `eventCampaignId === onChainCampaignId`.
 - Lookup `donorUserId` trong Mongo theo eventDonor (ví đã link với user nào).
12. Idempotent check — nếu `donationDAO.findByTxHash(txHash)` đã tồn tại, trả luôn doc cũ (FE retry an toàn).

13. `donationDAO.create({ campaignId, onChainCampaignId, donor: eventDonor, amountWei, txHash, ... })` — unique index `txHash` chống double-insert.
14. Đọc lại state on-chain
qua `_readChainState(onChainCampaignId) → campaignDAO.syncChainCache(...)` cập nhật `raisedWei`, `unlockedTotalWei`, `status`.
15. Nếu donor mới (`donationDAO.findOneByDonor` trước insert trả null), `campaignDAO.incrementDonorsCount(campaignId)` tăng `donorsCount`.
16. Response trả về frontend → modal show success + link Etherscan.

Cốt lõi của bước 11 là đoạn parse event đảm bảo backend chỉ tin những gì contract đã commit on-chain, không tin số liệu FE gửi:

```
// services/charityService.js - đoạn parse event Donated trong recordDonation
const contract = this._getReadOnlyContract();
const iface = contract.interface;
const eventTopic = iface.getEvent("Donated").topicHash;
const log = receipt.logs.find(
  (l) => l.topics[0] === eventTopic &&
    l.address.toLowerCase() === process.env.CHARITY_ADDRESS.toLowerCase()
);
if (!log) throw new AppError("Donated event not found in transaction", 400);

const parsed = iface.parseLog({ topics: log.topics, data: log.data });
const eventCampaignId = Number(parsed.args.id);
const eventDonor = parsed.args.donor.toLowerCase();
const eventAmountWei = parsed.args.amount.toString();

if (eventCampaignId !== Number(onChainCampaignId)) {
  throw new AppError("Event campaignId mismatch", 400);
}
```

Hình 30 Đoạn mã parse event để xác thực giao dịch on-chain

Kẻ tấn công có thể gửi `txHash` của một giao dịch khác (ví dụ giao dịch của người khác cùng số tiền) hòng ghi nhận mình là donor trong Mongo. Việc verify `eventDonor` từ event và so khớp với ví đã link với `userId` đảm bảo điều này không xảy ra: bản ghi Donation chỉ link với `donorUserId` nếu ví trên event khớp với ví đã link của user đó. Đây là lý do bước 11 trong sequence dài và phức tạp — định danh donor là mấu chốt cho luồng `claimRefund` về sau.

4.5.3 Off-chain cache pattern

Mục tiêu thiết kế: **mọi state quan trọng đều có một nguồn sự thật on-chain và một cache off-chain trong MongoDB**. On-chain là chậm, đắt, khó truy vấn phức tạp; off-chain nhanh, miễn phí, có index. Cache phải hội tụ về trạng thái on-chain trong thời gian ngắn.

Bảng 14 Mapping state on-chain và cache off-chain cho Charity

On-chain (Solidity)	Off-chain (Mongo)	Cách đồng bộ
campaigns[id].raised	Campaign.raisedWei (String BigInt)	Sau mỗi recordDonation, gọi <code>_readChainState(id)</code> rồi <code>syncChainCache(...)</code>
campaigns[id].unlockedTotal	Campaign.unlockedTotalWei	Sau unlockMilestone
campaigns[id].status	Campaign.status (OPEN/FUNDED/...)	Sau mọi tx ghi đổi state
campaigns[id].milestones[idx].unlocked	Campaign.milestones[idx].unlocked	Sau unlockMilestone
contributions[id][donor]	Donation.amountWei per donor	Append-only sau recordDonation, Donation.refundTxHash sau recordRefund

Quy tắc cache counter:

1. Counter wei không bao giờ tự cộng trong JS. Vì raisedWei là String BigInt, lệnh `parseInt + cộng + toString` chứa rất nhiều cạm bẫy (overflow Number, race condition giữa hai donation đồng thời). Mọi thay đổi raisedWei đều đọc lại từ chain (`contract.campaigns(id)` trả về raised mới nhất) rồi ghi đè qua `campaignDAO.syncChainCache`.
2. Counter donorsCount (Number) tăng qua `$inc`, nhưng phải check `donationDAO.findOneByDonor` trước để biết donor là mới hay cũ — không tăng trùng cho donation lần thứ hai của cùng một ví.

Trích đoạn `syncChainCache`:

```
// dao/campaignDAO.js
async syncChainCache(campaignId, chainState) {
  return Campaign.findByIdAndUpdate(campaignId, {
    raisedWei: chainState.raised.toString(),
    unlockedTotalWei: chainState.unlockedTotal.toString(),
    status: STATUS_NAMES[chainState.status], // mapping uint8 → string
  }, { new: true });
}
```

Hình 31 Trích đoạn hàm `syncChainCache` đồng bộ cache off-chain

Endpoint POST `/api/charity/campaigns/:id/sync` cho phép user chủ động kéo lại cache — hữu ích khi network chậm khiến cache trễ vài phút. Endpoint trả về snapshot trước/sau:

```
// charityService.syncFromChain
return {
  onChain: { raised: chain.raised.toString(), status: STATUS_NAMES[chain.status], ... },
  cache: { raised: campaign.raisedWei, status: campaign.status, ... },
  synced: true,
};
```

Hình 32 Endpoint POST `/api/charity/campaigns/:id/sync`

Frontend ở CharityDetail so sánh onChain vs cache rồi toast "đồng bộ khớp" hoặc "đã cập nhật cache do lệch". Mục đích thiết kế: giúp người dùng cuối tự verify rằng dữ liệu hiển thị trên trang khớp với chain — nâng tính minh bạch lên cấp người dùng cuối, không chỉ nhà phát triển.

Hệ thống được triển khai thực tế tại tên miền `medicine.id.vn` (có HTTPS) trên máy chủ EC2 của AWS dùng OS Ubuntu 22.04. Bảng dưới tổng hợp cấu hình triển khai.

Bảng 15 Cấu hình triển khai production

Hạng mục	Chi tiết
EC2	1vCPU, 1 GiB RAM, 20GB, Ubuntu 22.04
Tên miền	medicine.id.vn
Reverse proxy	Nginx 1.18, terminate TLS, forward <code>/api/*</code> → backend, <code>/*</code> → frontend build
HTTPS	Let's Encrypt qua Certbot, auto-renew
Backend process manager	PM2 với 1 instance (chuẩn bị sẵn cho cluster mode)
Frontend	Deploy trên Vercel
MongoDB	MongoDB Atlas free tier, region Singapore
Cloudinary	Free plan, region toàn cầu

Hạng mục	Chi tiết
Sepolia RPC	Alchemy free tier (300M compute units/tháng)

Cấu trúc URL công khai:

- <https://medicine.id.vn/> — frontend SPA.
- https://medicine.id.vn/api/* — backend REST.
- https://medicine.id.vn/socket.io/* — WebSocket (Nginx proxy có cấu hình proxy_set_header Upgrade \$http_upgrade).

Quy trình deploy mới một version:

1. Trên máy dev: chạy unit test cho contract (cd blockchain && npx hardhat test), commit, push lên main.
2. SSH vào AWS, git pull.
3. Backend: cd backend && npm ci && pm2 reload socialapp-backend.
4. Frontend: cd frontend && npm ci && npm run build — Nginx tự pick bản build mới vì trỏ tới frontend/build/.
5. Health-check: curl <https://medicine.id.vn/api/health> trả { success: true }.

Bảo mật triển khai:

- File .env của backend chỉ tồn tại trên VPS, không commit (đã có trong .gitignore).
- BE_WALLET_PRIVATE_KEY — private key của ví BE — chỉ chứa khoảng 0.5 Sepolia ETH (giá thực tế: 0 USD vì là testnet); ngay cả nếu key bị lộ, thiệt hại tối đa là kẻ tấn công chiếm dụng toàn bộ ETH trong ví và gọi ContentRegistry.registerPost với dữ liệu rác — không ảnh hưởng tài sản thực. Production thực sự sẽ phải dùng KMS / hardware wallet.
- Nginx có Strict-Transport-Security, X-Frame-Options: DENY, rate-limit ở tầng Nginx (bổ sung apiLimiter ở Express tầng app).

Việc triển khai thực tế thay vì chỉ demo trên localhost mang lại nhiều bài học mà localhost không bao giờ phơi bày: cấu hình WebSocket sticky session với Nginx, độ trễ của mạng Sepolia trên môi trường thật và sự biến động của Alchemy quota.

Chương 5 Kiểm thử và đánh giá

5.1 Kiểm thử Smart Contract

5.1.1 Phương pháp và công cụ

Toàn bộ smart contract được kiểm thử bằng framework Hardhat 2.22.0 kết hợp với thư viện assertion Chai 4.5.0 và Ethers.js 6.16.0. Các test chạy trên EVM in-process do Hardhat cung cấp với 20 tài khoản giả lập, không cần kết nối mạng thực — mỗi test mất vài chục mili-giây và có thể chạy hàng nghìn lần trong CI mà không tốn ETH thực.

```

blockchain/
├── contracts/
│   ├── ContentRegistry.sol
│   ├── Charity.sol
│   └── mocks/
│       └── ReentrancyAttacker.sol    ← contract giả lập tấn công
└── test/
    └── Charity.test.js              ← 40+ test case
  
```

Hình 33 Cấu trúc thư mục kiểm thử

```

cd blockchain
npx hardhat test                # chạy tất cả test
REPORT_GAS=true npx hardhat test # in báo cáo gas
npm run coverage                 # tính phần trăm code được cover
  
```

Hình 34 Câu lệnh chạy test

5.1.2 Phân nhóm các test case

Tổng cộng có trên 40 test case trong Charity.test.js, được tổ chức thành bảy nhóm describe, mỗi nhóm tương ứng với một khía cạnh của contract. Bảng dưới đây liệt kê đầy đủ.

Bảng 16 Phân nhóm và liệt kê test case của Charity.sol

Nhóm	Số test	Mục đích
Access control	5	Đảm bảo các function bị chặn khi gọi sai vai trò (non-whitelisted, non-admin, non-operator)

Nhóm	Số test	Mục đích
createCampaign	11	Happy path + 9 trường hợp validate input (sum, goal, duration, milestone count, milestone percent) + boundary 50/50
donate	5	Auto-transition OPEN → FUNDED khi đủ goal, các trường hợp revert (campaign không tồn tại, value = 0, sau deadline, đã FUNDED)
markFailed	3	Hết hạn không đủ goal chuyển FAILED, các revert case
claimRefund	3	Donor nhận đúng amount, double-claim revert, các trạng thái sai
Milestone flow	6	markExecuting + unlockMilestone + auto-transition EXECUTING → COMPLETED + revert double-unlock
adminForceFail	7	Admin force-fail khi dispute, các edge case (đã unlock milestone, đã fail, non-admin)
Reentrancy	2	Tấn công reentrancy vào claimRefund và donate — phải bị ReentrancyGuard chặn

5.1.3 Test reentrancy – điểm trọng tâm

Reentrancy attack là loại lỗ hổng nghiêm trọng nhất trong lịch sử Ethereum (The DAO 2016 — thiệt hại ~50 triệu USD). Đề tài thực hiện kiểm thử reentrancy bằng một mock contract chuyên dụng ReentrancyAttacker.sol, mô phỏng đúng cơ chế của attacker thật.

Mock contract sử dụng try/catch trong hàm receive() để re-entry vào claimRefund ngay khi nhận ETH gửi trả:

```

contract ReentrancyAttacker {
    Charity public charity;
    uint256 public attackCampaignId;
    uint256 public reentryCount;

    receive() external payable {
        // Cố gắng re-entry vào claimRefund khi nhận tiền refund.
        // try/catch để test không revert toàn bộ – chỉ kiểm tra
        // ReentrancyGuard có chặn được không.
        if (reentryCount < 2) {
            reentryCount++;
            try charity.claimRefund(attackCampaignId) {} catch {}
        }
    }
    // ... attack() function gọi donate + claimRefund
}

```

Hình 35 Mã nguồn ReentrancyAttacker.sol

Test xác nhận attacker chỉ rút được số tiền đúng bằng contributions[id][attacker] ban đầu, không thể nhân đôi nhờ re-entry — chứng minh modifier nonReentrant của OpenZeppelin hoạt động đúng:

```

it("claimRefund block reentrancy (attacker không nhận được gấp đôi)", async () => {
    // ... attacker donate 1 ETH, campaign FAILED
    const balanceBefore = await ethers.provider.getBalance(attacker.address);
    await attacker.attack(); // gọi claimRefund - receive() sẽ re-enter
    const balanceAfter = await ethers.provider.getBalance(attacker.address);

    expect(balanceAfter - balanceBefore).to.be.lte(parseEther("1.0"));
    // Attacker chỉ rút được tối đa 1 ETH đã donate, không hơn
});

```

Hình 36 Test Reentrancy

5.1.4 Báo cáo gas

Bảng dưới được trích từ REPORT_GAS=true npx hardhat test (gas cost trung bình các function, đơn vị: gas units).

Bảng 17 Gas cost trung bình các function Charity.sol

Function	Avg gas	Ước cost ở Sepolia (1 gwei × 3000 USD/ETH)
createCampaign (3 milestone)	~270.000	~\$0.81
donate	~95.000	~\$0.29

Function	Avg gas	Ước cost ở Sepolia (1 gwei × 3000 USD/ETH)
markExecuting	~52.000	~\$0.16
unlockMilestone	~70.000	~\$0.21
claimRefund	~58.000	~\$0.17
adminForceFail	~48.000	~\$0.14

Trên Sepolia testnet ETH miễn phí, nhưng số liệu này có giá trị tham khảo nếu sau này deploy lên mainnet hoặc Layer 2 (Arbitrum, Base)

5.1.5 Đánh giá độ bao phủ kiểm thử

Theo tiêu chí coverage của Hardhat (chạy `npm run coverage`):

- Statements coverage: ~95% — chỉ thiếu các nhánh không thể đạt được trong điều kiện bình thường (vd require đầu tiên đã chặn tất cả case sai trước khi vào nhánh thứ hai).
- Branch coverage: ~88% — các nhánh validate đều được test.
- Function coverage: 100% — mọi public function đều có ít nhất 1 test.

Bộ test này đủ để khẳng định contract hoạt động đúng spec ở mọi trạng thái hợp lệ và các cố gắng phá hoại phổ biến (sai vai trò, reentrancy, double claim, double unlock, deadline edge).

5.2 Tối ưu hiệu năng frontend

5.2.1 Phương pháp đo

Hệ thống được đo bằng Google Lighthouse (tích hợp trong Chrome DevTools) trên môi trường production thật tại `medicine.id.vn`, không phải localhost.

Cấu hình đo (đảm bảo có thể tái lập):

- Chrome chế độ ẩn danh — tắt cache + extension (đặc biệt MetaMask, vì extension này chiếm CPU và inject script vào trang).
- Mỗi page chạy 3 lần, lấy giá trị median từng metric (Lighthouse có dao động ± 5 điểm giữa các lần đo do biến thiên mạng và CPU).

- Throttling default: Slow 4G + 4× CPU slowdown cho mobile (giả lập máy Android tầm trung), no throttle cho desktop.
- Đo Performance category, các category còn lại (Accessibility, SEO, Best Practices) không trong phạm vi tối ưu của đợt này.

Hai trang quan trọng được đo:

- /charity — danh sách chiến dịch (list page với 6-12 card)
- /charity/:id — chi tiết chiến dịch (hero image + về tổ chức + danh sách donor + milestone list + nút donate)

5.2.2 Bottleneck phát hiện ở baseline

Kết quả đo baseline (trước tối ưu) cho thấy desktop rất tốt (Performance 95-99) nhưng mobile dưới chuẩn (~70). Bảng dưới tổng hợp số liệu mobile median 3 lần cho từng trang.

Bảng 18 Số liệu Lighthouse mobile (baseline — trước tối ưu)

Metric	/charity	/charity/:id	Mục tiêu Web Vitals
Performance	70	69	≥ 75
First Contentful Paint	1588 ms	1584 ms	< 1800 ms (đạt)
Largest Contentful Paint	6977 ms	7464 ms	< 2500 ms (vượt 3 lần)
Total Blocking Time	289 ms	313 ms	< 200 ms
Cumulative Layout Shift	0	0	< 0.1 (đạt)
Speed Index	1998 ms	1797 ms	< 3400 ms (đạt)

LCP cao bất thường — Lighthouse Opportunity tab chỉ ra hai nguyên nhân chính:

1. Total network payload 3.18 MB ở trang /charity mobile, trong đó có một ảnh Cloudinary nặng 1.2 MB đơn lẻ. Kiểm tra mã nguồn xác nhận URL ảnh được render thẳng từ database không qua bất kỳ transformation nào, nên browser tải nguyên file gốc (thường có resolution 2400×1600 trở lên dù container CSS chỉ cần ~400 px).

2. Unused JavaScript 750 KB — bundle ban đầu bao gồm cả các modal Web3 (DonateModal, ClaimRefundModal, TxStatusModal cùng dependency ethers) trong khi chỉ một phần nhỏ user thực sự sử dụng các modal này.

5.2.3 Các kỹ thuật tối ưu áp dụng

Ba nhóm tối ưu được áp dụng tuần tự:

P0 — Cloudinary URL transformation. Tạo helper mới src/utils/cloudinary.js và chèn chuỗi transformation `f_auto`, `q_auto`, `w_<width>`, `c_limit` vào URL Cloudinary ngay sau `/upload/`. Áp dụng tại bốn vị trí:

Bảng 19 Áp dụng Cloudinary URL transformation tại các vị trí render ảnh

Vị trí render <code></code>	Width transform	Bonus attribute
CampaignCard thumbnail	480 px	loading="lazy" (giữ nguyên)
CharityDetail hero (LCP element)	1200 px	fetchpriority="high"
Org logo	96 px	loading="lazy"
Donor avatar (lặp lại N lần)	64 px	loading="lazy"

Tham số `f_auto` cho phép Cloudinary tự chọn định dạng tối ưu (WebP/AVIF tùy browser hỗ trợ); `q_auto` chọn chất lượng nén thông minh; `c_limit` chỉ thu nhỏ ảnh nếu nó lớn hơn width đích (không upscale, giữ nguyên ảnh nhỏ). Helper trả về URL gốc không thay đổi nếu URL không phải Cloudinary (ví dụ avatar Google), bảo đảm an toàn cho mọi `<img>` trong project.

P1 — Lazy import modal Web3: Trong CharityDetail.jsx, hai modal DonateModal và ClaimRefundModal được chuyển từ import tĩnh sang React.lazy cộng với Suspense và render điều kiện:

```
const DonateModal = lazy(() =>
  import("../components/DonateModal/DonateModal"),
);

// ...

{donateModalOpen && (
  <Suspense fallback={null}>
    <DonateModal isOpen onClose={...} ... />
  </Suspense>
)}
```

Hình 37 Áp dụng lazy cho import DonateModel

Webpack tự tách hai modal này (cùng dependency ethers và TxStatusModal) thành một chunk JavaScript riêng có dung lượng ≈ 37 KB gzipped, chỉ được tải khi người dùng thực sự bấm nút "Ủng hộ" hoặc "Yêu cầu hoàn tiền". Initial bundle của trang chỉ tiết giảm tương ứng.

P2 — Network hint preconnect. Thêm hai dòng vào public/index.html

```
<link rel="preconnect" href="https://res.cloudinary.com" crossorigin />
<link rel="dns-prefetch" href="https://res.cloudinary.com" />
```

Hình 38 Network hint preconnect trong public/index.html

Browser sẽ thực hiện DNS-resolve và TLS handshake với Cloudinary CDN sớm, song song với quá trình parse HTML. Khi đến lượt browser fetch ảnh, kết nối đã sẵn sàng — tiết kiệm khoảng 100-200 ms trong giai đoạn đầu của LCP.

5.2.4 Kết quả sau tối ưu

Sau khi build production và deploy lên medicine.id.vn, đo lại Lighthouse với cùng cấu hình baseline. Bảng dưới so sánh trước và sau.

Bảng 20 Số liệu Lighthouse mobile (sau tối ưu)

Trang	Metric	Trước	Sau	Δ
/charity	Performance	70	79	+9 điểm
/charity	LCP	6977 ms	4362 ms	-2615 ms (-37%)
/charity	TBT	289 ms	268 ms	-21 ms

Trang	Metric	Trước	Sau	Δ
/charity	Total bytes	3.18 MB	< 800 KB	-2.4 MB (-75%)
/charity/:id	Performance	69	72	+3 điểm
/charity/:id	LCP	7464 ms	5253 ms	-2211 ms (-30%)
/charity/:id	TBT	313 ms	354 ms	+41 ms (dao động ± 50 ms)

Trang danh sách /charity cải thiện rõ rệt: Performance tăng 9 điểm và LCP giảm 37%, đưa trang về sát chuẩn "good" của Web Vitals.

Trang chi tiết /charity/:id cải thiện ít hơn (+3 điểm Performance) vì sau khi giải quyết được bottleneck ảnh, đường găng của LCP chuyển sang chuỗi tuần tự không tránh được trong mô hình SPA: parse main.js $\rightarrow$ React mount $\rightarrow$ fetch /api/charity/:id $\rightarrow$ setState $\rightarrow$ render .

Mỗi bước cộng dồn ~ 1 giây trên CPU mobile chậm (throttle 4 $\times$), tổng cộng vẫn ~ 5 giây dù ảnh đã nhẹ. Để giảm xuống dưới 2.5 giây cần kiến trúc Server-Side Rendering (Next.js / Astro).

5.3 Phản hồi từ Workshop review

5.3.1 Bối cảnh workshop

Ngày 2026-04-29, đề tài được trình bày tại buổi workshop nội bộ trước khi nộp báo cáo cuối kỳ. Các nhóm khác trong lớp đã review độc lập, mỗi nhóm chấm điểm và đưa nhận xét theo bảy tiêu chí của môn học: (1) chức năng, (2) tích hợp blockchain, (3) hiệu năng và minh chứng tối ưu, (4) kiến trúc hệ thống, (5) chất lượng mã nguồn, (6) bảo mật, (7) tài liệu nộp.

5.3.2 Tổng hợp phản hồi

Bảng 21 Phản hồi tích cực từ các nhóm review

Tiêu chí	Phản hồi tích cực
Tích hợp blockchain	Triển khai 2 contract trên Sepolia, có địa chỉ rõ ràng kèm Etherscan; phân biệt được pattern BE-relay (ContentRegistry) và FE-signed (Charity); Charity dùng AccessControl + ReentrancyGuard + CEI pattern + giải ngân theo milestone — thiết kế đúng best practice

Tiêu chí	Phản hồi tích cực
Kiến trúc	Tách rõ Controller / Service / DAO, error handling nhất quán bằng AppError, có winston logger, constants tập trung, cron job tách module, socket tách module
Chất lượng mã	Test coverage tốt; đặc biệt Charity.test.js có 40+ test case bao gồm reentrancy attack với mock contract; code đặt tên rõ ràng, có comment giải thích các quyết định thiết kế
Bảo mật	.env.example đầy đủ ba phần (BE/FE/blockchain), .gitignore cấu hình tốt, không lộ private key, có rate-limit + JWT + RBAC + input validation middleware
Phạm vi tính năng	"Phạm vi tính năng rất rộng" — feed nhiều tab, đăng bài, story, tìm kiếm, bạn bè, tin nhắn mã hóa, thông báo realtime, cộng đồng, quỹ từ thiện

Bảng 22 Bốn điểm trừ chung được nhắc đến

Điểm trừ	Mức độ ảnh hưởng
Thiếu báo cáo, slide thuyết trình, video demo tại thời điểm workshop	Quan trọng — không có gì để hộ kiểm tra
Thiếu file README.md ở thư mục gốc	Quan trọng — người mới khó tiếp cận project
Thiếu sơ đồ kiến trúc và data flow on-chain vs off-chain	Quan trọng — đánh giá kiến trúc khó nếu chỉ đọc code
Thiếu số liệu Lighthouse trước/sau tối ưu (chỉ có "sau")	Trung bình — không chứng minh được mức cải thiện

Bảng 23 Bug nghiêm trọng được phát hiện trong workshop

Bug	Mô tả
Chat duplicate	Một tin nhắn gửi đi hiển thị 2 lần — nghi do socket emit + optimistic update double
Google login lỗi	Đăng nhập bằng Google không hoạt động
i18n không cover Charity + Organizations	Vẫn hiện tiếng Việt khi switch sang tiếng Anh
On-chain stamp UX	Đăng bài có gọi registerPost nhưng không có feedback UI cho user
Đăng nhập bằng ví thất bại ở lần đầu	Lần đầu kết nối thất bại; sau khi đăng ký email, ví tự kết nối thành công ở lần thử kế tiếp — trải nghiệm người dùng gây nhầm lẫn
Charity desktop Lighthouse 60-68	Cần optimize và giải thích nguyên nhân
Mobile responsive yếu	Một số trang vỡ layout trên màn hình nhỏ
Chưa cho user đổi/xóa email liên kết	Thiếu tính năng quản lý tài khoản cơ bản

5.3.3 Các cải tiến đã thực hiện sau workshop

Sau khi nhận phản hồi từ workshop, nhóm đã thực hiện các cải tiến sau trong vòng một tuần:

Sửa các lỗi nghiêm trọng:

- Fix chat duplicate bằng cách bỏ optimistic update phía sender — chỉ render message khi nhận event message:new từ server (single source of truth).
- Fix Google login bằng cập nhật configure Firebase Admin SDK + verify idToken đúng cách phía BE.

- Bổ sung khoảng 60 i18n key cho charity.* và organization.*, kiểm thử switch ngôn ngữ trên cả hai trang.
- Thêm <TxStatusModal> cho luồng đăng bài on-chain — user thấy rõ tiến trình "đang ký", "đang xác nhận", "đã đóng dấu".
- Refactor flow login ví — auto-link wallet khi register email tồn tại nhưng có thông báo rõ ràng thay vì silent.
- Bổ sung tính năng đổi email liên kết và unlink wallet trong trang Settings.
- Cải thiện responsive mobile cho header, sidebar, charity detail và org detail (đo lại trên DevTools mobile emulator).

Bổ sung tài liệu:

- Viết README.md cho bốn vị trí: thư mục gốc, frontend/, backend/, blockchain/. Mỗi README hướng dẫn cài đặt local, env cần thiết, và lệnh chính.
- Vẽ sáu sơ đồ UML/diagram bằng draw.io bao gồm: kiến trúc hệ thống, kiến trúc phân tầng backend, ERD, sequence diagram cho ContentRegistry BE-relay, sequence diagram cho Charity Donate FE-signed, máy trạng thái Charity. Các sơ đồ này được nhúng trong Chương III và Chương IV.

Tối ưu hiệu năng:

- Đo Lighthouse 4 lần (hai trang × hai nền tảng desktop và mobile) trên môi trường sản xuất và lưu trữ kết quả tại baseline.
- Áp dụng ba nhóm tối ưu (Clouinary transform / Lazy modal / preconnect).
- Đo lại sau khi tối ưu và lưu trữ kết quả sau tối ưu để đối chiếu.

5.3.4 Tổng kết phản hồi workshop

Bốn điểm trừ chung và tám bug nghiêm trọng được phát hiện hoàn toàn nhờ quá trình review bên ngoài — những vấn đề mà nhóm phát triển khó tự nhận ra nếu chỉ kiểm thử nội bộ. Sau một tuần xử lý phản hồi, ba điểm trừ về tài liệu đã được giải quyết hoàn toàn (báo cáo, README, diagram, Lighthouse trước/sau) và toàn bộ tám bug đã được khắc phục.

5.4 Đánh giá blockchain

Để khẳng định blockchain trong đề tài không chỉ mang tính trang trí mà thực sự đem lại giá trị thiết thực, mỗi contract được đánh giá theo hai câu hỏi cốt lõi:

1. Minh bạch: Ai có thể tự kiểm chứng điều gì mà không cần tin server?
2. Hữu ích: Giá trị thực so với giải pháp Web2 thuần là gì?

5.4.1 ContentRegistry – Đóng dấu bài đăng

Tính minh bạch.

Contract ContentRegistry lưu trên Sepolia mapping postId → (contentHash, owner, timestamp). Bất kỳ ai có địa chỉ contract đều có thể gọi verifyPost(postId) từ Etherscan hoặc bất kỳ node Ethereum nào, không cần hỏi server SocialApp. Với một bài đăng cụ thể, một người ngoài có thể tự kiểm chứng:

- Bài đó đã được đăng tại block X, lúc thời điểm Y (không thể giả mạo vì timestamp của block là consensus-verified).
- Nội dung tại thời điểm đăng có content hash bằng đúng giá trị Z — nếu caption / ảnh / video bị sửa sau đó, hash mới sẽ khác và verify sẽ thất bại.

Đặc biệt, hash version 2 của đề tài nhúng authorId (user ID Mongo) vào payload trước khi tính keccak256. Điều này giải quyết một lỗ hổng nhỏ ban đầu của hash v1: kẻ tấn công có thể copy nguyên content của user khác và gọi registerPost từ ví của mình, sau đó tuyên bố "tôi đăng trước". Với v2, attacker không biết được authorId của victim sẽ đổi hash đầu vào, dẫn đến hash khác, không verify được dưới danh nghĩa người dùng thật. Phía BE và FE tự động chuyển đổi formula theo post.onChain.version để giữ tương thích ngược cho bài đăng cũ (null = v1, không migrate dữ liệu lịch sử).

Tính hữu ích.

So với Web2 thuần (timestamp lưu trong database), ContentRegistry giải quyết hai kịch bản tấn công không thể phòng ở Web2:

- Server bị xâm nhập: Hacker có quyền database có thể UPDATE timestamp của một bài để đẩy lùi thời gian đăng (giả mạo "tôi đăng trước"). Với ContentRegistry, timestamp được consensus của Ethereum xác nhận và không ai có thể sửa được (kể cả admin của hệ thống).
- Admin sửa nội dung lén: Admin xấu có thể UPDATE caption của một bài trong database mà không log audit. Người đọc không có cách nào phát hiện. Với ContentRegistry, content hash on-chain tố cáo ngay lập tức bất kỳ thay đổi nào — nút "Verify on-chain" trên giao diện gọi verifyPost và so sánh với content hiện tại.

Đây là một use-case nhỏ nhưng có giá trị thực, đặc biệt cho các nhóm cộng đồng cần lưu trữ tuyên bố hoặc hồ sơ bằng chứng có thể bị challenge sau này.

5.4.2 Charity – Quyên góp theo cột mốc

Tính minh bạch.

Contract Charity cho phép bất kỳ ai (kể cả không phải donor) kiểm tra trên Etherscan các thông tin sau cho từng campaign:

- Tổng số tiền đã quyên góp (raised) và tổng đã unlock cho org (unlockedTotal) — luôn cập nhật real-time.
- Số mốc đã unlock và số tiền của từng mốc (struct Milestone[campaignId][index]).
- Mọi giao dịch donate, unlock, claim refund — Etherscan list đầy đủ event log với địa chỉ donor, số tiền, block number.
- Vai trò AccessControl — ai có DEFAULT_ADMIN_ROLE, ai có OPERATOR_ROLE, ai được whitelist CAMPAIGN_CREATOR_ROLE.

Người dùng app cũng được hỗ trợ kiểm chứng nhanh thông qua bốn liên kết Etherscan trên trang chi tiết campaign (txHash createCampaign, contract address, transactions tab, events tab) cộng với nút **"Verify on-chain"** gọi BE so sánh cache Mongo với on-chain.

Tính hữu ích.

So với các nền tảng quyên góp Web2 (vd GoFundMe — đã có nhiều trường hợp lừa đảo được báo chí ghi nhận, nơi người tổ chức thu tiền rồi biến mất), Charity của SocialApp cung cấp ba bảo đảm cứng được enforce ở cấp contract:

- Tổ chức không thể tự rút tiền tùy ý. Tiền ETH bị khóa trong contract từ lúc donor chuyển vào. Org chỉ nhận được phần tiền của một mốc cụ thể khi OPERATOR_ROLE (ví BE, sau khi admin duyệt báo cáo off-chain) gọi unlockMilestone. Không có function "rút tất cả tiền" cho org.
- Donor luôn có thể đòi lại tiền nếu campaign thất bại. Khi markFailedIfExpired chuyển trạng thái sang FAILED (hết hạn không đủ goal) hoặc admin adminForceFail (có dispute), mỗi donor có thể tự gọi claimRefund(campaignId) để nhận lại đúng số tiền đã đóng. Pull-payment pattern này không đòi hỏi org phải hợp tác.
- Cấu trúc milestone không thể bị gian lận. Hardcode trong contract: campaign phải có ít nhất 2 milestone, không mốc nào quá 50% goal, tổng đúng bằng goal. Quy tắc này ngăn chặn một kịch bản lừa đảo phổ biến: đặt một cột mốc gom toàn bộ 100% số tiền mục tiêu, giải ngân xong rồi biến mất.

Defense-in-depth được thực hiện ở 3 layer (contract Solidity, BE service _validateAndParseCampaignPayload, FE form validate()), không tin layer nào hoàn toàn — kể cả khi BE bị bypass, contract vẫn từ chối campaign sai spec.

5.4.3 So sánh tổng quát với Web2

Bảng 24 So sánh giá trị cốt lõi giữa Web2 và blockchain trong đề tài

Yêu cầu	Giải pháp Web2 thuần	Giải pháp blockchain (đề tài)
Lưu timestamp đăng bài	DB row, tin admin	Block timestamp consensus-verified
Chống sửa nội dung lên	Audit log (tin admin không xoá)	Content hash on-chain, tự verify
Giữ tiền quyền góp	Stripe / bank, tin tổ chức	Smart contract khóa ETH, không ai rút được trừ theo rule
Hoàn tiền khi thất bại	Process thủ công, có thể bị từ chối	Donor tự claimRefund trên blockchain
Validate cấu trúc campaign	Server-side check (có thể bypass)	Solidity require không bypass được
Audit trail	DB log (admin xoá được)	Event log Ethereum (immutable)
Quyền điều hành	Admin trong DB	RBAC on-chain (AccessControl)

Đề tài không khẳng định blockchain là "ưu việt hơn" Web2 trong mọi tình huống — Web2 vẫn nhanh hơn, rẻ hơn và UX tốt hơn cho phần lớn use case của mạng xã hội (post, comment, friend, chat). Blockchain chỉ được áp dụng đúng chỗ: hai use case yêu cầu tính phi tập trung (đảm bảo tính toàn vẹn của bài đăng và tính minh bạch của quỹ từ thiện). Phần còn lại của hệ thống vẫn là MERN truyền thống.

Chương 6 Kết luận và Hướng phát triển

6.1 Kết quả đạt được

Đề tài đã hoàn thành việc xây dựng một website mạng xã hội tích hợp blockchain với đầy đủ các nhóm tính năng cốt lõi của một mạng xã hội hiện đại (đăng bài, bình luận, kết bạn, nhấn tin mã hóa, thông báo thời gian thực, nhóm cộng đồng) và hai mô-đun blockchain được lựa chọn có mục đích rõ ràng: ContentRegistry để đảm bảo tính toàn vẹn cho bài đăng, và Charity để quản lý quỹ quyên góp theo cột mốc một cách minh bạch.

Trên phương diện kỹ thuật, đề tài đã thực hiện được ba đóng góp đáng chú ý. Thứ nhất, áp dụng đồng thời hai pattern tích hợp blockchain khác biệt (BE-relay và FE-signed) với cùng một hệ thống, kèm theo phân tích so sánh và quy tắc lựa chọn pattern phù hợp với từng loại use case. Thứ hai, triển khai cơ chế bảo mật phòng vệ theo chiều sâu (defense in depth) qua ba tầng độc lập (smart contract, backend, frontend) đối với các quy tắc kinh doanh quan trọng. Thứ ba, xây dựng bộ kiểm thử gồm trên 40 test case cho smart contract Charity, trong đó có mô phỏng tấn công reentrancy bằng contract chuyên dụng nhằm xác minh tính hiệu quả của các biện pháp phòng vệ.

Hệ thống đã được triển khai thành công lên môi trường sản xuất (frontend trên Vercel, backend trên VPS với Nginx và PM2, smart contract trên Sepolia testnet) và đã trải qua một vòng workshop review với phản hồi từ bảy nhóm bên ngoài. Bộ phản hồi này đã được xử lý đầy đủ trong vòng một tuần, bao gồm sửa tám lỗi nghiêm trọng và bổ sung tài liệu, sơ đồ kiến trúc, kết quả đo hiệu năng Lighthouse trước và sau tối ưu.

6.2 Hạn chế

Mặc dù đã đạt được các mục tiêu đặt ra, đề tài vẫn còn một số hạn chế cần được nhìn nhận thẳng thắn. Trước hết, smart contract Charity chưa được kiểm toán bảo mật bởi bên thứ ba độc lập; bộ kiểm thử nội bộ và việc áp dụng các thư viện chuẩn của OpenZeppelin chỉ giảm thiểu rủi ro chứ chưa thể thay thế cho một quy trình audit chuyên nghiệp.

Thứ hai, hệ thống mới chỉ hoạt động trên Sepolia testnet — môi trường thử nghiệm với ETH miễn phí. Việc chuyển sang môi trường mainnet sẽ phát sinh chi phí gas thực tế và đặt ra các yêu cầu bổ sung về bảo mật khóa ví, giám sát giao dịch và xử lý các tình huống thất bại trên mạng lưới có giá trị thật.

Thứ ba, độ bao phủ kiểm thử chưa đồng đều giữa các phần của hệ thống. Smart contract đạt độ bao phủ cao (statements ~95%, function 100%) nhưng các tầng backend và frontend chủ yếu được kiểm thử thủ công, chưa có bộ kiểm thử tự động

đầy đủ. Tính năng nhắn tin mã hóa đầu cuối hiện tại sử dụng cơ chế đơn giản, chưa đạt mức độ bảo mật tương đương các giao thức chuyên dụng như Signal Protocol.

Cuối cùng, một số tính năng blockchain trong thiết kế ban đầu (token ERC-20 nội bộ, NFT thành viên, NFT bài đăng, chợ trao đổi vật phẩm) đã được chủ động loại khỏi phạm vi triển khai để bảo đảm chất lượng cho các tính năng đã có. Các tính năng này được dành lại cho các giai đoạn phát triển tiếp theo.

6.3 Hướng phát triển

Trên cơ sở các kết quả đã đạt được và những hạn chế đã nêu, đề tài đề xuất một số hướng phát triển trong tương lai như sau:

Về phương diện blockchain, hướng phát triển ưu tiên là kiểm toán bảo mật smart contract bởi đơn vị chuyên nghiệp trước khi triển khai lên mainnet, đồng thời nghiên cứu chuyển sang các giải pháp Layer 2 (Arbitrum, Base, Optimism) nhằm giảm chi phí gas và tăng tốc độ giao dịch. Bộ tính năng blockchain mở rộng cũng có thể được bổ sung, bao gồm cơ chế token nội bộ kết hợp tipping, NFT cho thành viên, NFT cho bài đăng có giá trị cao, và chợ trao đổi vật phẩm trong nhóm cộng đồng.

Về phương diện hệ thống, các hướng cải tiến đáng cân nhắc bao gồm xây dựng bộ kiểm thử tự động đầy đủ cho backend và frontend (unit test, integration test, end-to-end test), bổ sung hệ thống giám sát và cảnh báo (monitoring, alerting) cho môi trường sản xuất, và nâng cấp tính năng nhắn tin mã hóa lên giao thức bảo mật chuyên dụng.

Về phương diện trải nghiệm người dùng, đề tài có thể được mở rộng theo hướng hỗ trợ thêm các loại ví Web3 khác (WalletConnect, Coinbase Wallet) bên cạnh MetaMask, tích hợp tính năng đăng nhập không cần ví thông qua Account Abstraction (ERC-4337), và phát triển ứng dụng di động native song song với phiên bản web hiện tại.

Những hướng phát triển trên không chỉ giúp hoàn thiện sản phẩm mà còn mở ra cơ hội nghiên cứu chuyên sâu hơn về các chủ đề quan trọng trong lĩnh vực blockchain ứng dụng, từ tối ưu chi phí giao dịch đến trải nghiệm người dùng phi tập trung.

NGUỒN THAM KHẢO

- [1] [S. Kemp, “Digital 2025: Global Overview Report,” DataReportal, Feb. 5, 2025. Accessed: May 21, 2026.](#)
- [2] [MongoDB, “MERN Stack Explained.” Accessed: May 21, 2026.](#)
- [3] [Ethereum Foundation, “Ethereum Yellow Paper.” Accessed: May 21, 2026](#)
- [4] [Ethereum.org, “Ethereum Virtual Machine \(EVM\).” Accessed: May 21, 2026.](#)
- [5] [Solidity Documentation, “Introduction to Smart Contracts.” Accessed: May 21, 2026.](#)
- [6] [OpenZeppelin, “OpenZeppelin Contracts Documentation.” Accessed: May 21, 2026.](#)
- [7] [ethers.org, “Ethers v6 Documentation.” Accessed: May 21, 2026.](#)
- [8] [K. Finley, “A \\$50 Million Hack Just Showed That the DAO Was All Too Human,” Wired, Jun. 18, 2016. Accessed: May 21, 2026.](#)
- [9] [Solidity Documentation, “Security Considerations.” Accessed: May 21, 2026.](#)
- [10] [M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token \(JWT\),” RFC 7519, IETF, May 2015. Accessed: May 21, 2026.](#)